



大语言模型及应用

Instructor: Xiaohan Ma(马晓寒), Ph.D.

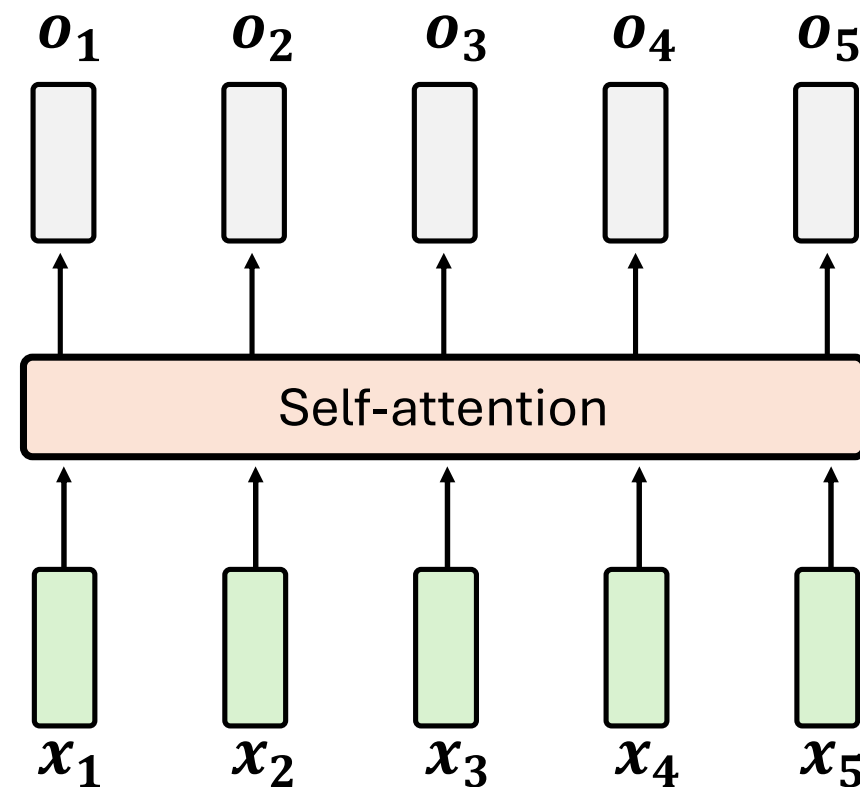
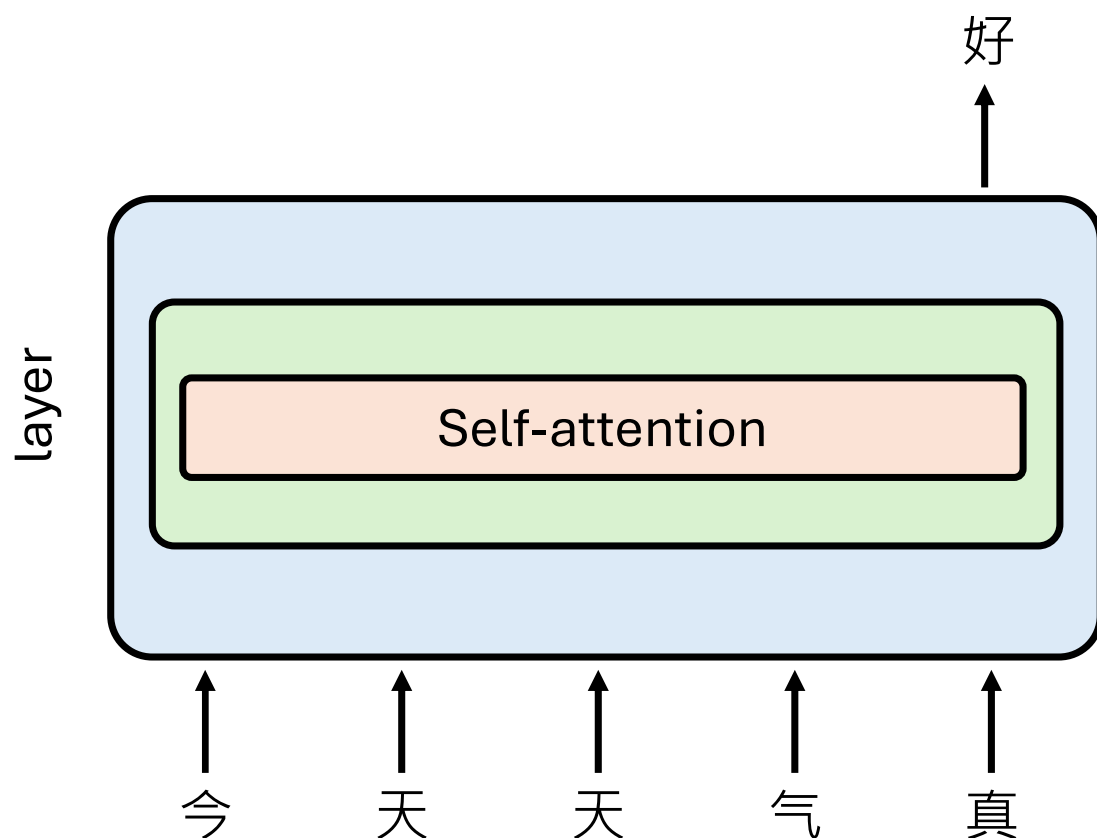
Graduate School of Information and Communication Technology,

Ajou University, Korea

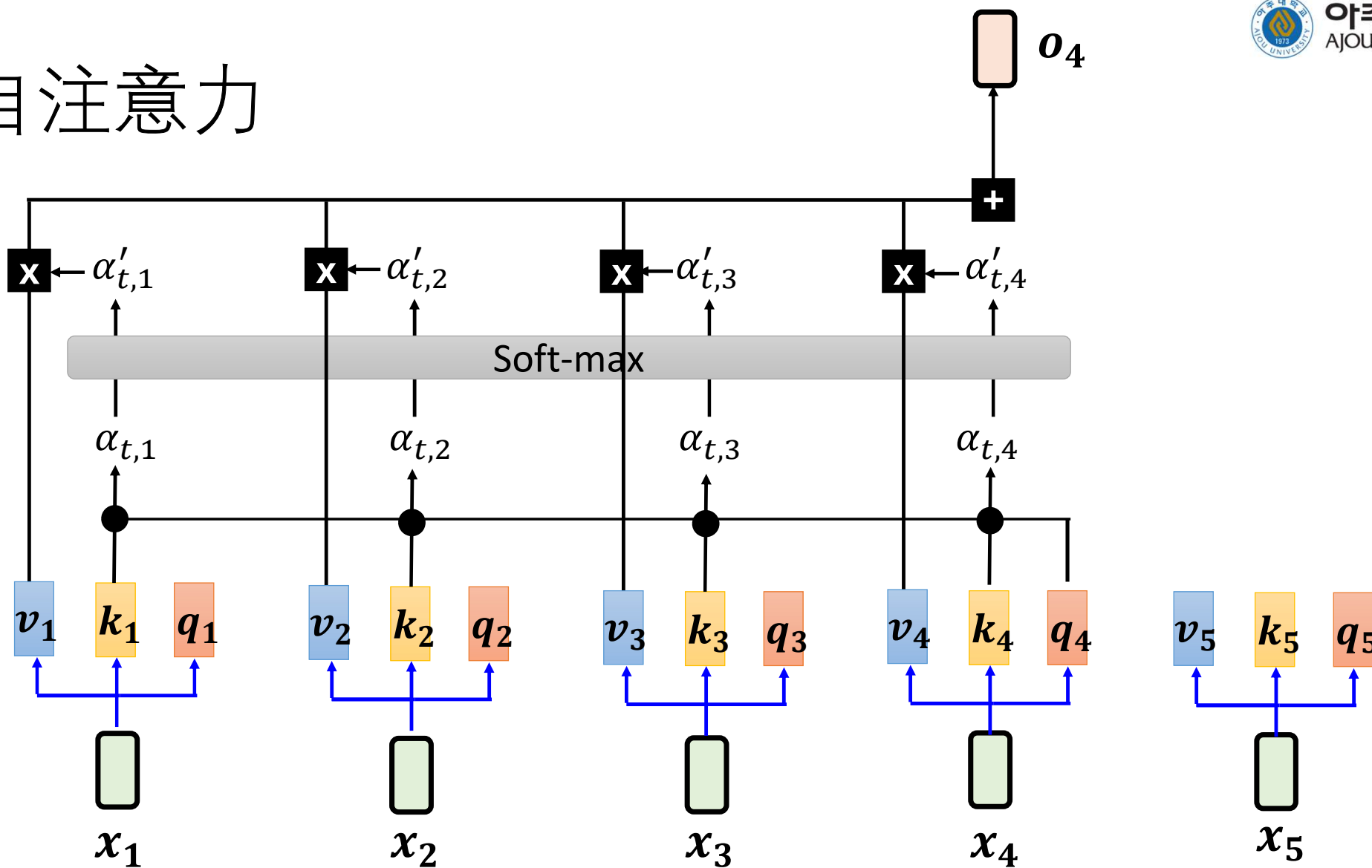
maxiaohan@ajou.ac.kr

现代大模型结构优化

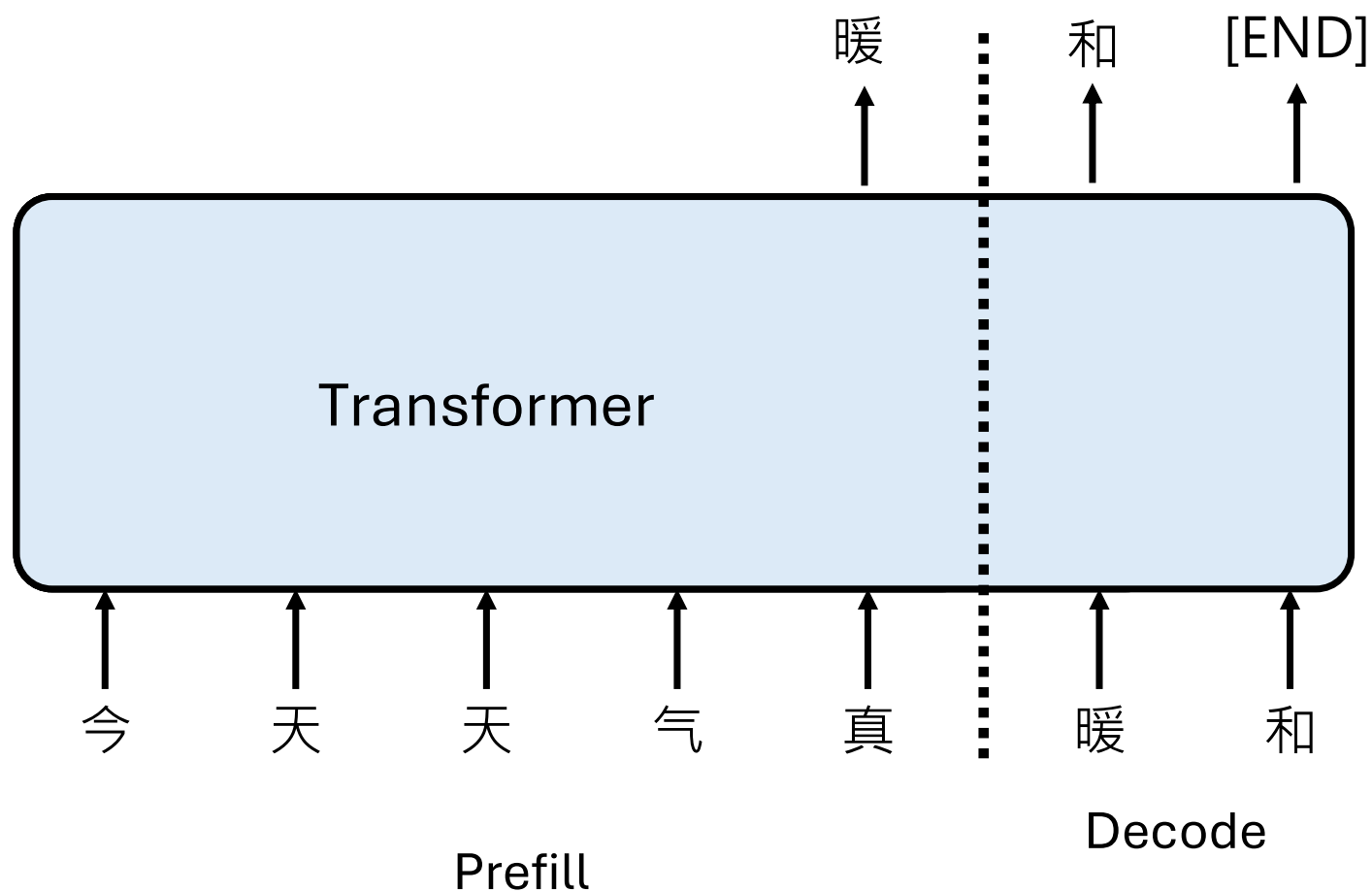
语言模型如何生成 (推论, Inference)



自注意力



语言模型如何生成 (推論, Inference)



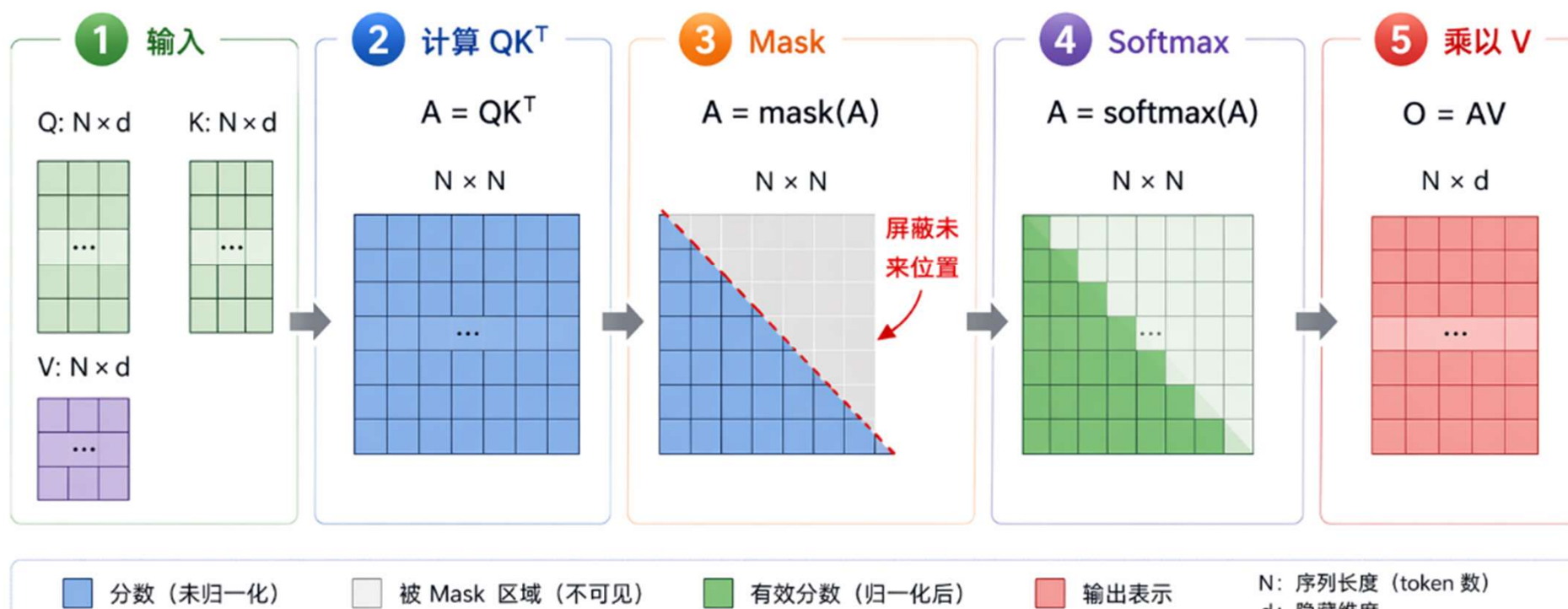


Flash Attention

<https://arxiv.org/abs/2205.14135>

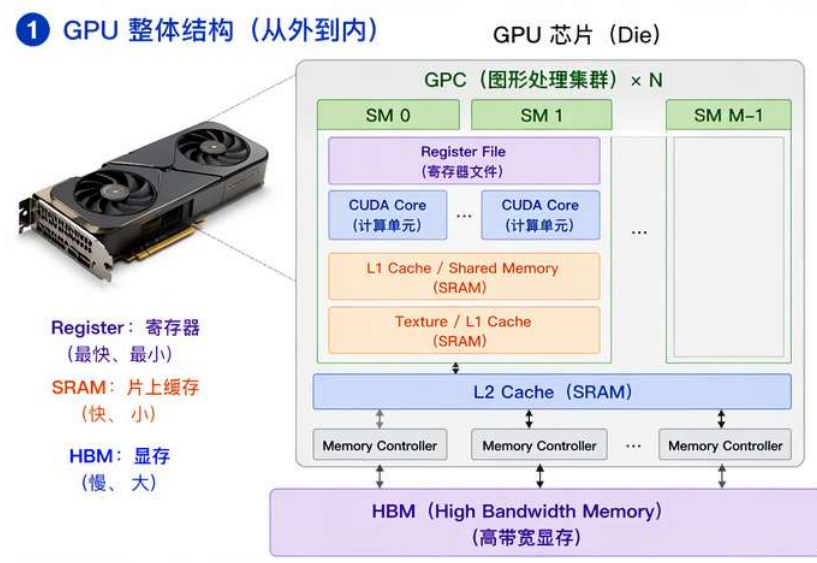
Attention 在GPU上的计算流程

目标：计算 $O = \text{Softmax}(QK^T) V$



GPU整体结构

- GPU采用分层存储结构：越靠近计算单元越快，越远容量越大
- 三种存储位置
- **Register（寄存器）**
线程私有，速度最快，用于临时变量
- **SRAM（片上缓存）**
多个线程共享，容量小，速度快
- **HBM（显存）**
容量最大，速度最慢，存模型参数与输入输出



HBM 与 SRAM: 容量 vs 速度

HBM 存储大量物品但取用慢, SRAM 只能放少量但取用快

HBM (高带宽内存)

大容量, 慢速度



容量大

可以存放大量物品 (数据)



速度慢

取用需要时间 (访问延迟高)



成本低

单位存储成本低, 容量扩展容易



适合场景

存放大量不常用的数据

取数据 (读)

从 HBM 取出数据
放到 SRAM 处理



存结果 (写)

处理完成后
再写回 HBM



SRAM (静态随机存储器)

容量小, 速度快



容量小

只能放少量物品 (数据)



速度快

取用几乎瞬间完成 (访问延迟低)



成本高

单位存储成本高, 容量扩展困难



适合场景

存放当前正在使用的关键数据



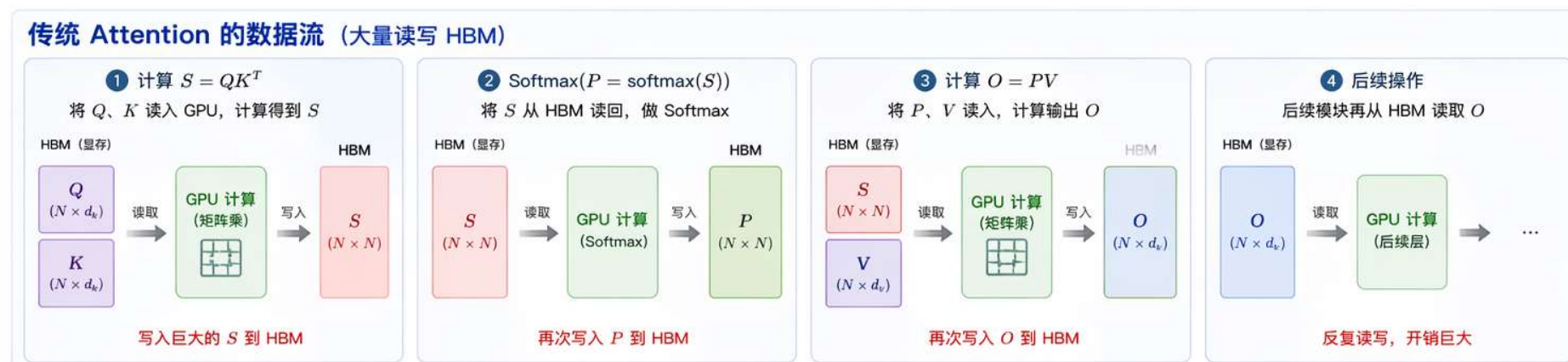
核心理念

将大量数据存放在 HBM 中, 需要时搬运到 SRAM 中快速处理, 处理完成再写回 HBM。

用空间换时间, 充分发挥两种存储的优势!

传统的Attention计算为什么慢?

- 为什么慢?
 - 大量的HBM读写
 - 以N=4096为例
 - Score 矩阵S大小: $N \times N = 4096 \times 4096 = 16,777,216$ 元素
 - 如果用 FP16 (2 bytes): 约 32 MB
 - 一个模型有多个Head、多个Layer, 多个Batch: 显存占用迅速膨胀!



HBM 与 SRAM: 与 Attention 计算的关系

Attention 需要频繁读写大量中间矩阵（如 K、V、Score、P），传统方法反复访问 HBM，导致速度慢；
将数据放在 SRAM 中复用，可显著提升计算效率。

传统方式：频繁访问 HBM（慢）

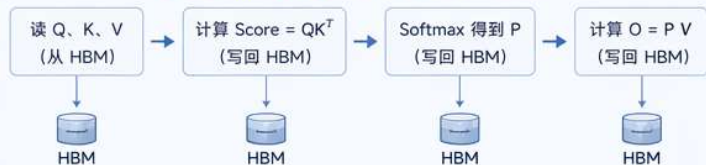
每步都要从 HBM 读写大量数据



反复读写

大量数据在 HBM
和计算单元之间
来回搬运

传统 Attention 计算流程（以单个 Head 为例）



低效原因

- ✗ Q、K、V、Score、P、O 等大矩阵频繁读写 HBM
- ✗ HBM 带宽有限，数据搬运成为瓶颈
- ✗ 大量时间浪费在等待内存，GPU 计算单元利用率低

Attention 核心计算

$$\text{Score} = QK^T / \sqrt{d_k}$$

$$P = \text{Softmax}(\text{Score})$$

$$O = P V$$

$$Q: (N_q \times d_k)$$

$$K: (N_k \times d_k)$$

$$V: (N_k \times d_v)$$

$$O: (N_q \times d_v)$$

关键点

Attention 的三步
计算需要多次复用
K、V 和中间结果 P。
把这些数据放在
SRAM 中，减少
对 HBM 的访问，
才能高效完成计算！

优化方式：数据驻留 SRAM（快）

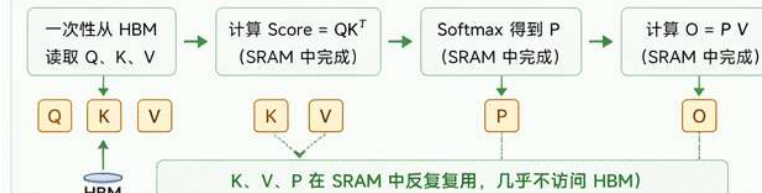
将数据放入 SRAM，在片上快速复用



数据常驻

在 SRAM 中快速
读写和复用，
几乎不访问 HBM

优化后的 Attention 计算流程（以单个 Head 为例）



高效原因

- ✓ 数据驻留 SRAM，复用 K、V、P，减少大规模数据搬运
- ✓ 片上带宽高、延迟低，计算单元始终有数据可用
- ✓ GPU 利用率高，整体速度显著提升

Flash Attention

- 核心思想
 - 不存储整个 $N \times N$ 的自注意力权重(score)矩阵, 分块计算,
 - 在线Softmax, 边算边输出
 - 在读取过程中, 不断更新全局最大值 m 和指数和 s , 最终得到与原softmax完全一致的结果

Softmax: 从数学定义到工程实现

- 目标: 把任意实数向量 x 的分数(score)转换为概率分布 (非负且和为1)

1. 数学定义 (Theory)

给定向量 $x = [x_1, x_2, \dots, x_n]$

Softmax 定义为:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

- 输出为概率: $0 < \text{softmax}(x_i) < 1$
- 所有输出之和为 1: $\sum_i \text{softmax}(x_i) = 1$
- 单调性: x_i 越大, $\text{softmax}(x_i)$ 越大

例子 $x = [2, 1, 4, 0]$ (一次性计算)

x_i	2	1	4	0
e^{x_i}	7.389	2.718	54.598	1.000
$e^{x_i} / \sum_j e^{x_j}$	0.119	0.044	0.880	0.016

$$\sum_j e^{x_j} = 65.705$$

Softmax的工程实现示例

例子：假设一行 score 为

2	1	4	0
---	---	---	---

Softmax 的定义：

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \Rightarrow = \frac{e^{x_i-m}}{\sum_j e^{x_j-m}}$$

其中 $m = \max_j x_j$ (用于数值稳定)

1 找到最大值 m

$$m = \max(2, 1, 4, 0) = 4$$

用最大值来做平移，
可以避免指数爆炸。

2 计算每个元素的指数 (减去最大值 m)

x_i	2	1	4	0
$x_i - m$	$2 - 4 = -2$	$1 - 4 = -3$	$4 - 4 = 0$	$0 - 4 = -4$
e^{x_i-m}	$e^{-2} = 0.1353$	$e^{-3} = 0.0498$	$e^0 = 1$	$e^{-4} = 0.0183$



我们实际计算的是 e^{x_i-m} ，
这样数值更稳定。

3 计算分母 (指数和)

$$s = \sum_j e^{x_j-m} = 0.1353 + 0.0498 + 1 + 0.0183 = 1.2034$$

分母 s 是所有指数的总和，
这是 Softmax 的归一化因子。

4 计算每个位置的 Softmax 概率

$p_i = \frac{e^{x_i-m}}{s}$	位置 i	1	2	3	4
	x_i	2	1	4	0
	e^{x_i-m}	0.1353	0.0498	1	0.0183
	概率 p_i	$0.1353/1.2034$ $= 0.112$	$0.0498/1.2034$ $= 0.041$	$1/1.2034$ $= 0.831$	$0.0183/1.2034$ $= 0.015$

最终 Softmax 概率 (和为 1)

位置 1  0.112
位置 2 
位置 3  0.831
概率 4  0.015

检查: $0.112 + 0.041 + 0.831 + 0.015 = 0.999 \approx 1$ ✓

Online Softmax 计算图解

1 把数据分两块

原始数据 (4 个 score)



2 分块读取顺序



3 初始化 (在读取任何数据之前)

全局最大值

$m = -\infty$

全局指数和

$s = 0$



解释:

还没有读取任何数据,
所以最大值设为 $-\infty$,
指数和设为 0 作为起点。

Online Softmax 计算图解

目标：逐块读取数据，处理当前块时，更新全局最大值 m 和指数和 s 。

当前处理：第一块 [2, 1]

1 输入数据（第一块）

原始数据分两块：

2	1	4	0
---	---	---	---

当前处理：第一块 [2, 1]

初始化（在读任何数据之前）

- 全局最大值 $m = -\infty$
- 指数和 $s = 0$

2 处理当前块 [2, 1]

当前块数据 x

2	1
---	---

计算块内最大值

$$m_{block} = \max(2, 1) = 2$$

3 更新全局最大值 m

旧的全局最大值

$$m = -\infty$$

新的全局最大值

$$\begin{aligned} m &= \max(m, m_{block}) \\ &= \max(-\infty, 2) = 2 \end{aligned}$$

为什么这样更新？

- 因为指数函数单调递增，使用更大的最大值可以保证数值稳定。

4 计算指数并更新指数和 s

对当前块每个元素计算

$e^{x_i - m}$ （使用新的全局 $m = 2$ ）

x_i	2	1
$x_i - m$	0	-1
$e^{x_i - m}$	$e^0 = 1$	$e^{-1} = 0.3679$

更新指数和 s

$$\begin{aligned} s &= s + \sum e^{x_i - m} \\ &= 0 + (1 + 0.3679) \\ &= 1.3679 \end{aligned}$$

5 当前保存的状态（处理完第一块后）

当前保存的全局状态

全局最大值

$$m = 2$$

指数和

$$s = 1.3679$$

💡 要点

- 只依赖当前块数据 [2, 1]
- 无需知道后续块
- 得到更新后的 m 和 s ，用于处理下一块

Online Softmax



目标：处理第二块数据 $[4, 0]$ ，在保持数值稳定的同时，更新全局最大值 m 和指数和 s 。

当前状态（来自第3页处理完第一块后的结果）

全局最大值 $m = 2$

指数和 $s = 1.3679$



注意：旧的 s 是按旧的最大值 2 算的，现在最大的是 4，需要把旧的 s 换算到新的基准 4 下。

1 输入第二块数据

第二块数据：

4 0

2 计算块内最大值

块内最大值：

$$m_{block} = \max(4, 0) = 4$$

3 更新全局最大值 m

新的全局最大值：

$$\begin{aligned} m_{new} &= \max(m, m_{block}) \\ &= \max(2, 4) = 4 \end{aligned}$$

4 缩放旧的 s 到新基准（从 2 $\rightarrow$ 4）

旧的 s 是按基准 2 算的，需要换到基准 4。

换算公式： $s_{old} \times e^{m - m_{new}}$

旧的 s 缩放到新基准 4：

$$s_{old} \times e^{2-4} = 1.3679 \times e^{-2} = 0.1851$$

5 计算第二块在新基准 4 下的指数和

对第二块每个元素计算 $e^{x_i - m_{new}}$ ($m_{new} = 4$)

x_i	4	0
$x_i - m_{new}$	0	-4
$e^{x_i - m_{new}}$	$e^0 = 1$	$e^{-4} = 0.0183$

第二块的指数和：

$$s_{new_block} = e^0 + e^{-4} = 1 + 0.0183 = 1.0183$$

6 更新全局指数和 s

把旧的 s （已缩放）与新块的指数和相加：

$$\begin{aligned} s_{new} &= (s_{old} \times e^{2-4}) + s_{new_block} \\ &= 0.1851 + 1.0183 = 1.2034 \end{aligned}$$

7 当前保存的状态（处理完第二块后）

全局最大值
 $m = 4$

指数和
 $s = 1.2034$

★ 要点

- 更新 m 为新的最大值 4
- 旧的 s 缩放到新基准 4
- 加上新块的指数和
- 得到新的 $s = 1.2034$



目标： 所有块都处理完后，利用全局最大值 m 和指数和 s ，计算最终的 Softmax 概率并输出结果。

回顾：
两块处理后的
全局状态

处理第一块 $[2, 1]$ 后
 $m = 2, s = 1.3679$



处理第二块 $[4, 0]$ 后 (最终状态)
 $m = 4, s = 1.2034$

最终全局状态 (所有块处理完毕)

全局最大值
 $m = 4$

全局指数和
 $s = 1.2034$

1 利用全局 m 和 s ，计算每个元素的 Softmax

$\text{Softmax}(x_i) = \frac{e^{x_i - m}}{s}$ ，其中 $m = 4, s = 1.2034$

输入 x_i	2	1	4	0
计算 $x_i - m$	$2 - 4 = -2$	$1 - 4 = -3$	$4 - 4 = 0$	$0 - 4 = -4$
计算 $e^{x_i - m}$	$e^{-2} = 0.1353$	$e^{-3} = 0.0498$	$e^0 = 1.0000$	$e^{-4} = 0.0183$
$\text{Softmax}(x_i) = \frac{e^{x_i - m}}{s}$	$\frac{0.1353}{1.2034} = 0.1125$	$\frac{0.0498}{1.2034} = 0.0414$	$\frac{1.0000}{1.2034} = 0.8309$	$\frac{0.0183}{1.2034} = 0.0152$

★ 要点

- 使用最终的全局最大值 $m = 4$ 进行归一化，保证数值稳定性；
- 所有概率之和为 1；
- 结果与一次性计算 Softmax 完全一致。

2 验证： 概率和为 1

概率之和

$0.1125 + 0.0414 + 0.8309 + 0.0152 = 0.9999 (\approx 1)$

✓ 数值误差仅来自浮点计算，理论上等于 1。

3 最终 Softmax 输出结果

2 1 4 0

Softmax

Softmax(x)

0.1125

0.0414

0.8309

0.0152

Online Softmax vs Softmax

1 原始数据与普通 Softmax (一次性全部读取)

原始数据 x :

2	1	4	0
---	---	---	---

普通 Softmax 的计算过程:

① 全局最大值: $m = \max(2, 1, 4, 0) = 4$

② 指数和:

$$s_{full} = \sum e^{x_i - m} = e^{2-4} + e^{1-4} + e^{4-4} + e^{0-4} = e^{-2} + e^{-3} + e^0 + e^{-4} = 1.2034$$

③ 最终概率: $P_i = \frac{e^{x_i - m}}{s_{full}}$

x_i	2	1	4	0
$e^{x_i - m}$	e^{-2} 0.1353	e^{-3} 0.0498	e^0 1.0000	e^{-4} 0.0183
P_i	0.1125	0.0414	0.8309	0.0152

2 Online Softmax (分块读取) 的过程回顾

将数据分成两块: 第一块 [2, 1], 第二块 [4, 0]

处理第一块 [2, 1]

块内数据

2	1
块内最大值 2	

块内最大值

新全局最大值 $m = \max(-\infty, 2) = 2$

指数和

$$s = e^{2-2} + e^{1-2} = 1 + e^{-1} = 1.3679$$

处理第二块 [4, 0]

块内数据

4	0
块内最大值 4	

块内最大值

新全局最大值 $m_{new} = \max(2, 4) = 4$

指数和更新

$$\begin{aligned} s_{new} &= s \cdot e^{m_{old} - m_{new}} + \sum e^{x_i - m_{new}} \\ &= 1.3679 \times e^{2-4} + (e^{4-4} + e^{0-4}) \\ &= 0.1851 + 1.0183 = 1.2034 \end{aligned}$$

处理完所有块后的最终状态: 全局最大值 $m = 4$, 指数和 $s = 1.2034$

Online Softmax vs Softmax

3 发现了吗？指数和 s 与一次性计算完全一致

普通 Softmax 的指数和：

$$s_{full} = 1.2034$$

=

Online Softmax 的指数和：

$$s_{new} = 1.2034$$

为什么会相等？（核心原因）

- 每次遇到更大的最大值时，我们把“旧的指数”按比例缩放： $s_{old} \times e^{m_{old} - m_{new}}$
- 这等价于把旧的每一项 $e^{x_i - m_{old}}$ 转换到新的基准 m_{new} 下：

$$e^{x_i - m_{old}} \times e^{m_{old} - m_{new}} = e^{x_i - m_{new}}$$

- 因此，最终的 s 就是所有元素都减去最终最大值 m 的指数之和：

$$s = \sum e^{x_i - m} = e^{2-4} + e^{1-4} + e^{4-4} + e^{0-4} = 1.2034$$

- 与一次性计算完全一致！

4 最终概率完全一样

因为 Online Softmax 得到的 (m, s) 与普通 Softmax 完全相同 ($m = 4$, $s = 1.2034$)，所以每个位置的概率： $P_i = \frac{e^{x_i - m}}{s}$ 也完全相同。

x_i	2	1	4	0
$x_i - m (m=4)$	-2	-3	0	-4
$e^{x_i - m}$	e^{-2} 0.1353	e^{-3} 0.0498	e^0 1.0000	e^{-4} 0.0183
除以 $s = 1.2034$ $P_i = \frac{e^{x_i - m}}{s}$	0.1125	0.0414	0.8309	0.0152

Flash Attention 例子

- 通过分块读取K,V，边计算边更新，最终得到与一次性完全一样的输出

问题设定：我们要计算一个最小版 Attention

Query (只有1个)

$$q = [1]$$

我们只有 1 个 Query，
维度是 1 (为了简单起见)。

Keys (共有4个)

$$K = [2, 1, 4, 0]$$

4 个 Key，
每个都是 1 维。

Values (共有4个)

$$V = [10, 20, 30, 40]$$

每个 Key 都对应
一个 Value (1 维)。

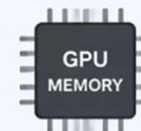
计算目标

我们要计算 Attention 输出：

$$o = \text{Softmax}(qK^T)V$$

并且不用一次性把所有数据读进内存。

为什么要分块？



真实模型中， N_k (Key 的个数)
非常大 (可能是几千、几万)。
一次性加载所有 K 、 V 会爆显存，
所以要分块 (block) 处理。

分块处理方式 (本例子分成两块)

第一块 (Block 1)

$$K^{(1)} = [2, 1]$$

$$V^{(1)} = [10, 20]$$



按顺序
逐块读取

第二块 (Block 2)

$$K^{(2)} = [4, 0]$$

$$V^{(2)} = [30, 40]$$

普通Attention的计算流程

1 计算所有 score (一次性计算)

$$Q = [1] \quad K = [2, 1, 4, 0]$$

score = qK (1 维相乘, 等价于点积)

$$\text{score} = [1 \times 2, 1 \times 1, 1 \times 4, 1 \times 0] = [2, 1, 4, 0]$$

2 计算 Softmax (一次性计算)

最大值 $m = 4$ score - $m = [-2, -3, 0, -4]$

$$e^{\text{score}-m} = [e^{-2}, e^{-3}, e^0, e^{-4}]$$

$$\approx [0.1353, 0.0498, 1, 0.0183]$$

分母 (指数和)

$$s = 0.1353 + 0.0498 + 1 + 0.0183$$

$$= 1.2034$$

$$\text{Softmax 概率 } p_i = \frac{e^{\text{score}_i - m}}{s}$$

位置 i	1	2	3	4
概率 p_i	0.1125	0.0414	0.8309	0.0152

3 计算输出 (一次性计算)

Values

$$V = [10, 20, 30, 40]$$

$$o = \sum_{i=1}^4 p_i v_i$$

$$= 0.1125 \times 10 + 0.0414 \times 20 + 0.8309 \times 30 + 0.0152 \times 40$$

$$= 1.125 + 0.828 + 24.927 + 0.608$$

$$= 27.4889 \approx 27.49$$

标准答案

Flash Attention 计算流程-1



Block 1 的计算图解 (处理前 2 个位置)

给定: $q_1 = 1$

$$K = [2, 1, 4, 0]$$

$$V = [10, 20, 30, 40]$$

本次处理: Block 1 (前 2 个位置)

$$k_1 = 2, k_2 = 1$$

$$v_1 = 10, v_2 = 20$$

目标: 计算处理完 Block 1 后的状态 (m_1, s_1, o_1)

- m_1 : 当前最大 score (标量)
- s_1 : 指数和 (分母, 标量)
- o_1 : 输出 (加权和, 标量)

① 更新最大值 m_1

计算 scores

$$score_i = \langle q_1, k_i \rangle = q_1 k_i$$

i	k_i	$score_i = q_1 k_i$
1	2	$1 \times 2 = 2$
2	1	$1 \times 1 = 1$

取最大值

$$m_1 = \max(score_1, score_2)$$

$$= \max(2, 1) = 2$$

$$m_1 = 2$$

② 更新指数和 s_1

按当前最大值 m_1 计算指数

$$e^{score_i - m_1}$$

i	$score_i$	$score_i - m_1$	$e^{score_i - m_1}$
1	2	$2 - 2 = 0$	$e^0 = 1$
2	1	$1 - 2 = -1$	e^{-1}

求和 (指数和)

$$\begin{aligned} s_1 &= \sum_{i=1}^2 e^{score_i - m_1} \\ &= e^0 + e^{-1} \\ &= 1 + e^{-1} \end{aligned}$$

$$s_1 = 1 + e^{-1} \approx 1.3679$$

③ 更新输出 o_1

计算加权贡献

$$\frac{e^{score_i - m_1}}{s_1} v_i$$

i	$e^{score_i - m_1}$	$\frac{e^{score_i - m_1}}{s_1}$	v_i	贡献
1	1	$\frac{1}{1 + e^{-1}}$	10	$\frac{10}{1 + e^{-1}}$
2	e^{-1}	$\frac{e^{-1}}{1 + e^{-1}}$	20	$\frac{20e^{-1}}{1 + e^{-1}}$

求和 (输出)

$$\begin{aligned} o_1 &= \sum_{i=1}^2 \frac{e^{score_i - m_1}}{s_1} v_i \\ &= \frac{10}{1 + e^{-1}} + \frac{20e^{-1}}{1 + e^{-1}} \\ &= \frac{10 + 20e^{-1}}{1 + e^{-1}} \end{aligned}$$

$$o_1 = \frac{10 + 20e^{-1}}{1 + e^{-1}} \approx 12.6894$$

Flash Attention 计算流程-2

Block 2 的计算图解 (第 1 页: 计算 m_2 和 s_2)

已知: 处理完 Block 1 后的状态

$$(m_1, s_1, o_1) = \left(2, 1 + e^{-1}, \frac{10 + 20e^{-1}}{1 + e^{-1}} \right)$$

本次处理: Block 2 (后 2 个位置)

$$k_3 = 4, \quad k_4 = 0 \\ v_3 = 30, \quad v_4 = 40$$

目标: 更新最大值 m_2 和指数和 s_2

- m_2 : 新的最大 score (标量)
- s_2 : 所有已处理位置的指数和 (分母, 标量)

① 更新最大值 m_2

① 计算 Block 2 的 scores

$$score_i = \langle q_1, k_i \rangle = q_1 k_i$$

i	k_i	$score_i = q_1 k_i$
3	4	$1 \times 4 = 4$
4	0	$1 \times 0 = 0$

② 与当前最大值比较, 更新最大值

$$m_2 = \max(m_1, score_3, score_4) \\ = \max(2, 4, 0) \\ = 4$$



解释:

- m_1 是到目前为止 (Block 1) 的最大 score;
- 本块 (Block 2) 中最大 score 为 4;
- 因此新的最大值 $m_2 = 4$ 。

$$m_2 = 4$$

② 更新指数和 s_2

FlashAttention 递推公式:

$$s_2 = s_1 e^{m_1 - m_2} + \sum_{i=3}^4 e^{score_i - m_2}$$

① 旧部分: 将 Block 1 的贡献重标定

$$s_1 e^{m_1 - m_2} \\ = (1 + e^{-1}) e^{2-4} \\ = (1 + e^{-1}) e^{-2} \\ = e^{-2} + e^{-3}$$

+

② 新部分: 加入 Block 2 的贡献

$$\sum_{i=3}^4 e^{score_i - m_2} \\ = e^{score_3 - 4} + e^{score_4 - 4} \\ = e^{4-4} + e^{0-4} \\ = 1 + e^{-4}$$

③ 得到新的指数和 s_2

$$s_2 = (e^{-2} + e^{-3}) + (1 + e^{-4}) \\ = 1 + e^{-2} + e^{-3} + e^{-4}$$

$$s_2 = 1 + e^{-2} + e^{-3} + e^{-4} \approx 1.2034$$

Block 2 的计算图解 (第 2 页: 计算输出 o_2)

Flash Attention 计算流程-2

已知: 来自第 1 页的结果

$$(m_1, s_1, o_1) = \left(2, 1 + e^{-1}, \frac{10 + 20e^{-1}}{1 + e^{-1}} \right)$$

本次处理: Block 2 (后 2 个位置)

$$\begin{aligned} \text{scores: } & \text{score}_3 = 4, \text{ score}_4 = 0 \\ \text{values: } & v_3 = 30, v_4 = 40 \end{aligned}$$

目标: 更新输出 o_2

- 使用 FlashAttention 递推公式:

$$o_2 = o_1 \frac{s_1}{s_2} e^{m_1 - m_2} + \sum_{i=3}^4 \frac{e^{\text{score}_i - m_2}}{s_2} v_i$$

③ 更新输出 o_2 —— 按公式分两部分计算

Part A: 旧块 (Block 1) 的贡献

公式:

$$o_1 \frac{s_1}{s_2} e^{m_1 - m_2}$$

代入已知量:

$$\left(\frac{10 + 20e^{-1}}{1 + e^{-1}} \right) \cdot \frac{1 + e^{-1}}{1 + e^{-2} + e^{-3} + e^{-4}} \cdot e^{2-4}$$

化简:

$$\frac{10 + 20e^{-1}}{1 + e^{-2} + e^{-3} + e^{-4}} \cdot e^{-2}$$

展开为指数形式:

$$\frac{10e^{-2} + 20e^{-3}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

$$\text{旧块贡献} = \frac{10e^{-2} + 20e^{-3}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

Part B: 新块 (Block 2) 的贡献

公式:

$$\sum_{i=3}^4 \frac{e^{\text{score}_i - m_2}}{s_2} v_i$$

i	score_i	$\text{score}_i - m_2$	$e^{\text{score}_i - m_2}$	v_i	$\frac{e^{\text{score}_i - m_2}}{s_2} v_i$
3	4	$4 - 4 = 0$	$e^0 = 1$	30	$\frac{1}{1 + e^{-2} + e^{-3} + e^{-4}} \cdot 30$
4	0	$0 - 4 = -4$	e^{-4}	40	$\frac{e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}} \cdot 40$

相加得到新块贡献:

$$\frac{30 + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

$$\text{新块贡献} = \frac{30 + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

合并两部分得到最终输出 o_2

$$o_2 = \text{Part A} + \text{Part B}$$

$$o_2 = \frac{10e^{-2} + 20e^{-3}}{1 + e^{-2} + e^{-3} + e^{-4}} + \frac{30 + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

合并分子:

$$o_2 = \frac{30 + 10e^{-2} + 20e^{-3} + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}}$$

$$o_2 = \frac{30 + 10e^{-2} + 20e^{-3} + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}} \approx 27.49$$



直观理解:

- 分母 s_2 是所有位置 (1~4) 的指数和;
- 分子是所有位置的 $e^{\text{score}_i - m_2}$ 乘以对应 v_i 的和;
- 这与一次性 softmax 的加权输出完全一致。



本页小结 (计算输出 o_2)

按 FlashAttention 递推公式, 将输出分为两部分:

- 旧块贡献: 需要用 $e^{m_1 - m_2}$ 和 $\frac{s_1}{s_2}$ 进行重标定;
- 新块贡献: 直接用新最大值 m_2 计算权重并加入。

至此, 处理完 Block 2 后的状态:

$$(m_2, s_2, o_2) = \left(4, 1 + e^{-2} + e^{-3} + e^{-4}, \frac{30 + 10e^{-2} + 20e^{-3} + 40e^{-4}}{1 + e^{-2} + e^{-3} + e^{-4}} \right)$$

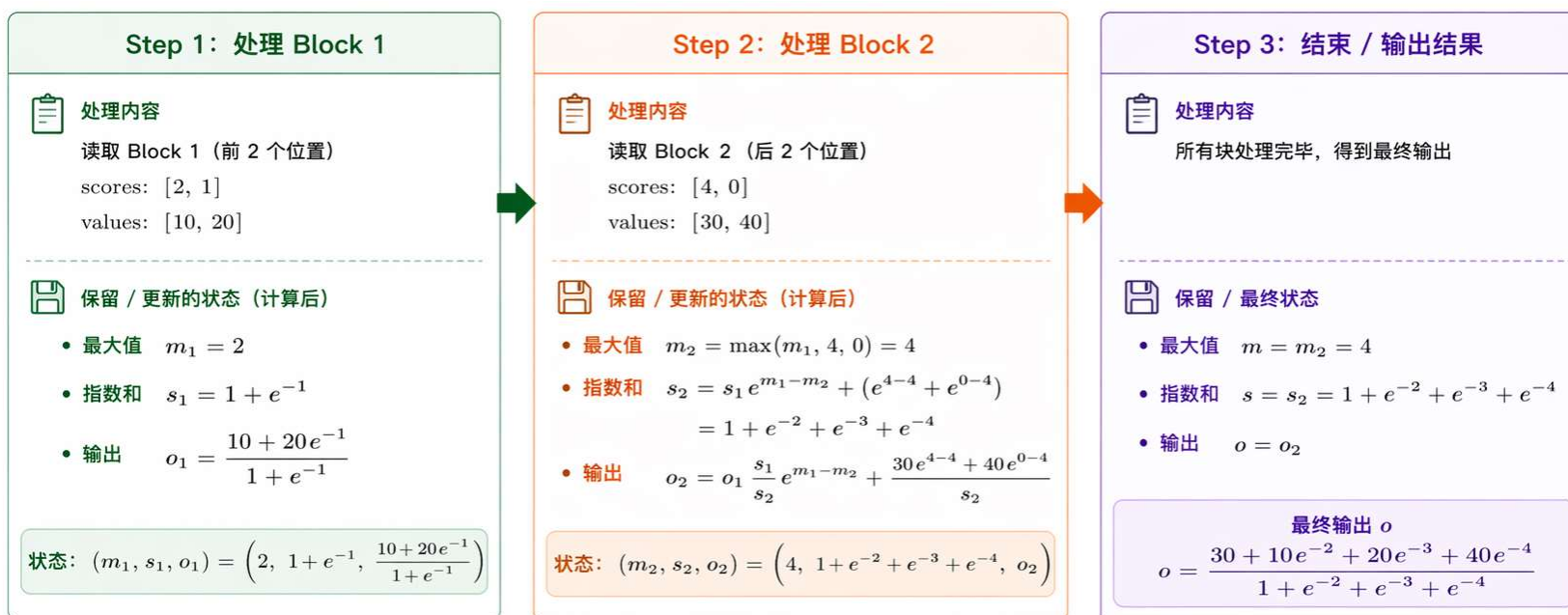
下一步:

- 若还有后续 Block, 继续用递推公式更新。
→ (m_3, s_3, o_3)

Flash Attention

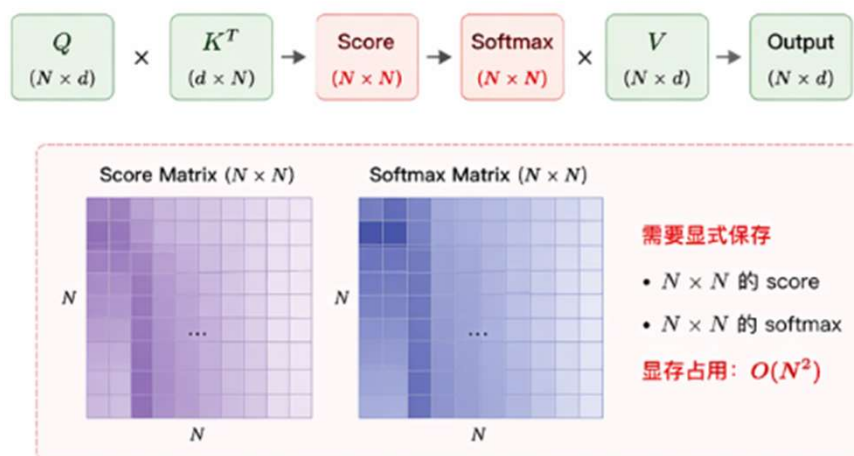
• 特点

- 不需要把所有的K,V和注意力权重(score)都读入内存
- 只需分块读取，边读边更新，显著减少GPU显存占用
- 计算结果与普通Attention完全一致，数值稳定
- 改变的是计算顺序和存储方式，没有改变任何数学公式。

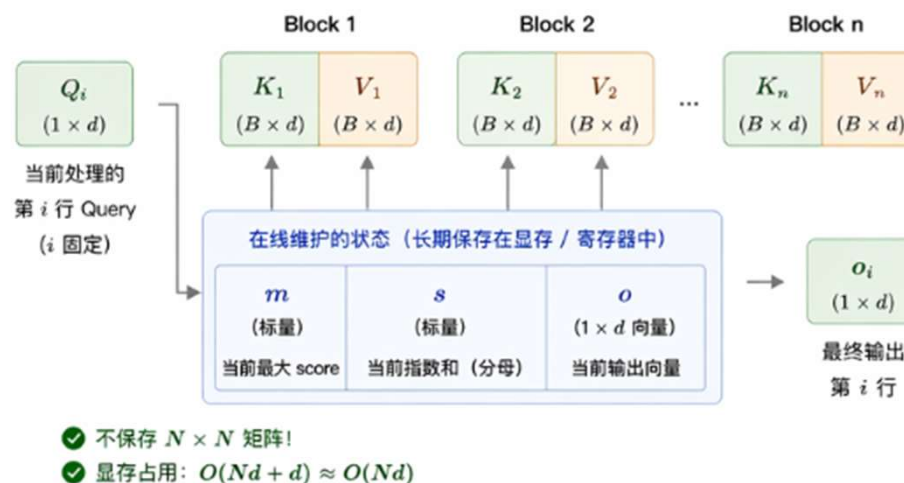


Attention VS Flash Attention

1 普通 Attention: 需要保存大量矩阵 (显存压力大)



2 FlashAttention: 只保存少量状态 (显存友好)



1. 处理第一个块 (Block 1) : 初始化状态 m_1, s_1, o_1



我们按块 (block) 读取数据, 首先处理第一个块 (Block 1)。
我们仅考虑查询向量 q_1 (第 1 个位置的 query)。
Block 1 包含前 N 个位置的 k, v 向量。

查询
 q_1 (第 1 个位置)

q_1

数据分块示意 (仅显示与 q_1 相关的部分)

	Block 1 (本页处理)	Block 2 (下一页处理)
K (键)	$k_1 \quad k_2 \quad \cdots \quad k_N$	$k_{N+1} \quad k_{N+2} \quad \cdots \quad k_{2N}$
V (值)	$v_1 \quad v_2 \quad \cdots \quad v_N$	$v_{N+1} \quad v_{N+2} \quad \cdots \quad v_{2N}$

Block 1 的输入数据 (针对查询 q_1)

- 查询向量 (固定): $q_1 \in \mathbb{R}^{d_k}$
- 键 (Block 1): $k_1, k_2, \dots, k_N \in \mathbb{R}^{d_k}$
- 值 (Block 1): $v_1, v_2, \dots, v_N \in \mathbb{R}^{d_v}$



注意: 我们以查询 q_1 为例进行说明,
其他查询的计算过程相同。



目标: 基于 Block 1, 计算并初始化状态 (m_1, s_1, o_1) 。
这些状态将用于后续块的递推更新。

从 Block 1 计算初始状态 (m_1, s_1, o_1) (以查询 q_1 为例)

① 计算 scores (相似度)

先计算 q_1 与 Block 1 中所有键 k_j 的 scores:

$$score_j = \langle q_1, k_j \rangle = q_1^T k_j, \quad j = 1, 2, \dots, N$$

记 scores 向量为:

$$[score_1, score_2, \dots, score_N] \in \mathbb{R}^{1 \times N}$$

作用: 得到查询 q_1 对 Block 1 中
所有位置的原始相似度分数。

② 指数和 s_1

先取最大值:

$$m_1 = \max_{1 \leq j \leq N} score_j$$

计算基于 m_1 的指数和:

$$s_1 = \sum_{j=1}^N e^{score_j - m_1}$$

作用: 对应 softmax 的分母
(未归一化的总权重)。

③ 输出 o_1

计算归一化加权后的输出:

$$o_1 = \sum_{j=1}^N \frac{e^{score_j - m_1}}{s_1} v_j$$

作用: 对应 softmax 加权后的
输出 (已归一化)。

初始化后的状态 (处理完 Block 1)

m_1 : 当前已见 scores 的最大值 (标量)
 s_1 : 基于 m_1 的指数和 (标量)
 o_1 : 基于 m_1 的加权输出 (向量, 维度为 d_v)



状态向量

$$\begin{bmatrix} m_1 \\ s_1 \\ o_1 \end{bmatrix}$$

将用于
下一块的
递推更新!



直观理解:

- m_1 就是当前看到的“最高分”;
- s_1 是所有位置相对 m_1 的“总权重”;
- o_1 是按这些权重计算的“平均结果”。



小结: 处理完第一个块后, 我们得到了状态 (m_1, s_1, o_1) 。在下一页, 我们将利用这些状态,
加入第二个块 $(k_{N+1}, \dots, k_{2N}, v_{N+1}, \dots, v_{2N})$, 更新得到新的状态 (m_2, s_2, o_2) 。

2. 处理第二个块 (Block 2) : 更新状态 m_2, s_2, o_2



继续处理第二个块 (Block 2), 包含下一批 N 个位置的 k, v 。
我们在保持数值稳定的前提下, 将 Block 2 的贡献并入已有状态, 得到新的状态 m_2, s_2, o_2 。

数据分块示意 (仅显示与 q_1 相关的部分)

Block 1 (已处理, 状态为 m_1, s_1, o_1)

K (键)

k_1

k_2

$\cdots$

k_N

V (值)

v_1

v_2

$\cdots$

v_N

Block 2 (本页处理)

k_{N+1}

k_{N+2}

$\cdots$

k_{2N}

v_{N+1}

v_{N+2}

$\cdots$

v_{2N}

查询向量

$q_1 \in \mathbb{R}^{dk}$

q_1

Block 2 的输入数据 (针对查询 q_1)

- 查询向量 (固定): $q_1 \in \mathbb{R}^{dk}$
- 键 (Block 2): $k_{N+1}, k_{N+2}, \dots, k_{2N} \in \mathbb{R}^{dk}$
- 值 (Block 2): $v_{N+1}, v_{N+2}, \dots, v_{2N} \in \mathbb{R}^{dv}$



注意: 我们仍使用 q_1 与本块所有键计算 scores, 并将结果与已有状态 (m_1, s_1, o_1) 合并更新。



目标: 在处理完 Block 2 后, 得到新的状态 m_2, s_2, o_2 , 这些状态将用于后续块的递推更新。

从状态 (m_1, s_1, o_1) 处理 Block 2, 得到新的状态 (m_2, s_2, o_2) (以查询 q_1 为例)

① 计算 Block 2 的 scores

对每个位置 $j = N+1, N+2, \dots, 2N$, 计算 score:

$$score_j = \langle q_1, k_j \rangle = q_1^T k_j$$

$$j = N+1, N+2, \dots, 2N$$

记 scores 向量为:
 $[score_{N+1}, score_{N+2}, \dots, score_{2N}] \in \mathbb{R}^{1 \times N}$

作用: 得到查询 q_1 与 Block 2 中所有键的原始相似度分数。

② 更新最大值 m_2 和指数和 s_2

先更新最大值:

$$m_2 = \max \left(m_1, \max_{N+1 \leq j \leq 2N} score_j \right)$$

更新指数和 (分母的未归一化总权重):

$$s_2 = s_1 \cdot e^{m_1 - m_2} + \sum_{j=N+1}^{2N} e^{score_j - m_2}$$

作用: 将 Block 1 的指数和按新最大值 m_2 重标定, 并加上 Block 2 的指数和, 得到新的总权重。

③ 更新输出 o_2

先计算 Block 2 的加权输出 (分子贡献):

$$num_2 = \sum_{j=N+1}^{2N} e^{score_j - m_2} v_j$$

更新输出:

$$o_2 = o_1 \cdot \frac{s_1}{s_2} \cdot e^{m_1 - m_2} + \frac{num_2}{s_2}$$

作用: 将 Block 1 的输出按新权重重标定, 并加上 Block 2 的归一化加权输出。

得到新的状态 (处理完 Block 2)

m_2 : 截至当前, 所有已处理位置中的最大 score (标量)
 s_2 : 截至当前, 所有已处理位置的指数和 (标量)
 o_2 : 截至当前, 基于 softmax 的加权输出 (向量, 维度为 d_v)



状态向量

$$\begin{bmatrix} m_2 \\ s_2 \\ o_2 \end{bmatrix}$$

将在下一页用于处理后续块!



直观理解:

- m_2 是目前看到的所有位置中的“最高分”;
- s_2 是所有位置相对 m_2 的“总权重”;
- o_2 是按这些权重计算的“平均结果”。



小结: 处理完第二个块后, 我们得到了状态 (m_2, s_2, o_2) 。在下一页, 我们将继续处理第三个块 $(k_{2N+1}, \dots, k_{3N}, v_{2N+1}, \dots, v_{3N})$, 重复相同的更新过程。

3. 处理第 t 个块 (Block t , $t \geq 3$): 通用更新公式 m_t, s_t, o_t



继续处理第 t 个块 (Block t), 包含位置 $j = (t-1)N+1, \dots, tN$ 的 k, v 。
在保持数值稳定的前提下, 将该块的贡献并入已有状态 $(m_{t-1}, s_{t-1}, o_{t-1})$,
得到新的状态 (m_t, s_t, o_t) 。

数据分块示意 (仅显示与 q_1 相关的部分)

	Block 1 (已处理)	Block 2 (已处理)	...	Block t (本页处理)	Block T (后续)	查询向量 $q_1 \in \mathbb{R}^{dk}$
K (键)	$k_1 \dots k_N$	$k_{N+1} \dots k_{2N}$		$k_{(t-1)N+1} \dots k_{tN}$	$k_{(T-1)N+1} \dots k_{TN}$	q_1
V (值)	$v_1 \dots v_N$	$v_{N+1} \dots v_{2N}$		$v_{(t-1)N+1} \dots v_{tN}$	$v_{(T-1)N+1} \dots v_{TN}$	

Block t 的输入数据 (针对查询 q_1)

- 查询向量 (固定): $q_1 \in \mathbb{R}^{dk}$
- 键 (Block t): $k_{(t-1)N+1}, \dots, k_{tN} \in \mathbb{R}^{dk}$
- 值 (Block t): $v_{(t-1)N+1}, \dots, v_{tN} \in \mathbb{R}^{dv}$



注意: 我们仍使用 q_1 与本块所有键计算 scores,
并将结果与已有状态 $(m_{t-1}, s_{t-1}, o_{t-1})$ 合并更新。



目标: 在处理完第 t 个块后, 得到新的状态 (m_t, s_t, o_t) ,
这些状态将用于后续块的递推更新。

从状态 $(m_{t-1}, s_{t-1}, o_{t-1})$ 处理 Block t , 得到新的状态 (m_t, s_t, o_t) (以查询 q_1 为例)

① 计算 Block t 的 scores

对每个位置 $j = (t-1)N+1, \dots, tN$,
计算 score:

$$score_j = \langle q_1, k_j \rangle = q_1^T k_j$$

$$j = (t-1)N+1, \dots, tN$$

记 scores 向量为:

$$[score_{(t-1)N+1}, score_{(t-1)N+2}, \dots, score_{tN}] \in \mathbb{R}^{1 \times N}$$

作用: 得到查询 q_1 与 Block t 中所有键
的原始相似度分数。

② 更新最大值 m_t 和指数和 s_t

先更新最大值:

$$m_t = \max(m_{t-1}, \max_{(t-1)N+1 \leq j \leq tN} score_j)$$

更新指数和 (分母的未归一化总权重):

$$s_t = s_{t-1} \cdot e^{m_{t-1}-m_t} + \sum_{j=(t-1)N+1}^{tN} e^{score_j-m_t}$$

作用: 将已处理部分的指数和按新最大值 m_t 重标定,
并加入 Block t 的指数和, 得到新的总权重。

③ 更新输出 o_t

先计算 Block t 的加权输出 (分子贡献):

$$num_t = \sum_{j=(t-1)N+1}^{tN} e^{score_j-m_t} v_j$$

更新输出:

$$o_t = o_{t-1} \cdot \frac{s_{t-1}}{s_t} \cdot e^{m_{t-1}-m_t} + \frac{num_t}{s_t}$$

作用: 将已处理部分的输出按新权重重标定,
并加入 Block t 的归一化加权输出。

得到新的状态 (处理完 Block t)

m_t : 截至当前, 所有已处理位置中的最大 score (标量)
 s_t : 截至当前, 所有已处理位置的指数和 (标量)
 o_t : 截至当前, 基于 softmax 的加权输出 (向量, 维度为 d_v)



状态向量

$$\begin{bmatrix} m_t \\ s_t \\ o_t \end{bmatrix}$$

将用于
下一块的
递推更新!



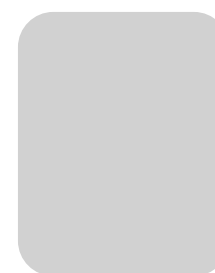
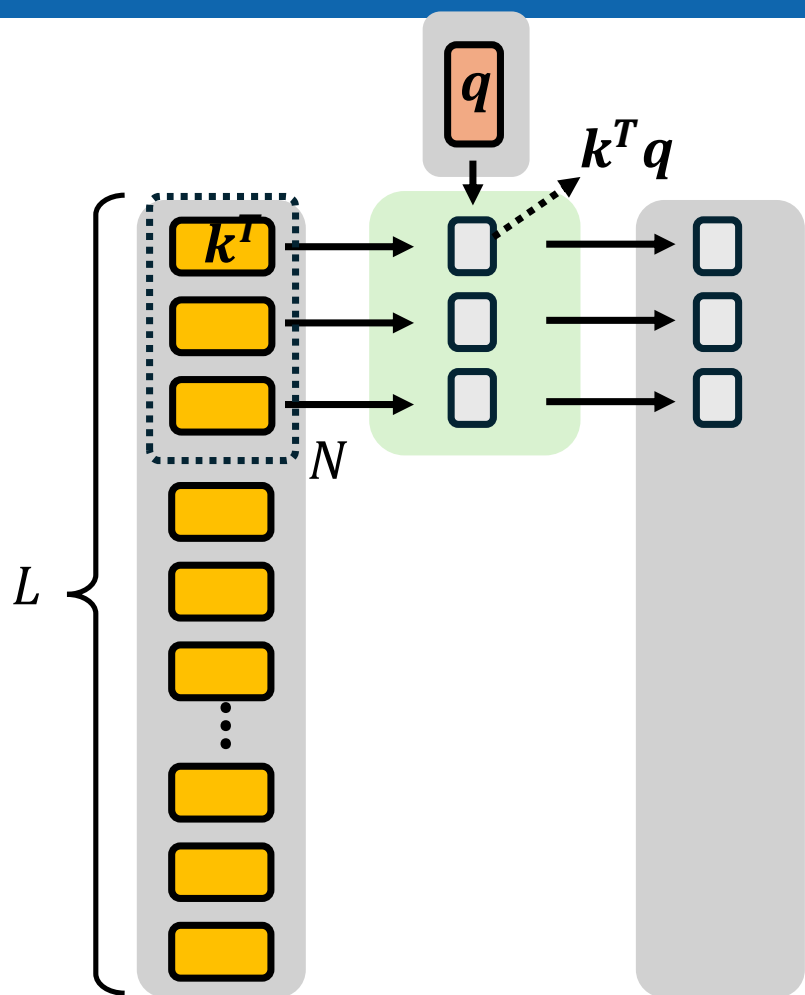
直观理解:

- m_t 是目前看到的所有位置中的“最高分”;
- s_t 是所有位置相对 m_t 的“总权重”;
- o_t 是按这些权重计算的“平均结果”。

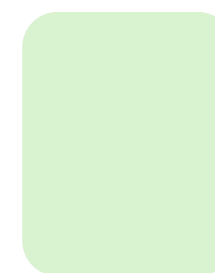


小结: 处理完第 t 个块后, 我们得到了状态 (m_t, s_t, o_t) 。在下一页, 我们将继续处理第 $t+1$ 个块
($k_{tN+1}, \dots, k_{(t+1)N}, v_{tN+1}, \dots, v_{(t+1)N}$), 重复相同的更新过程, 直到处理完所有位置。

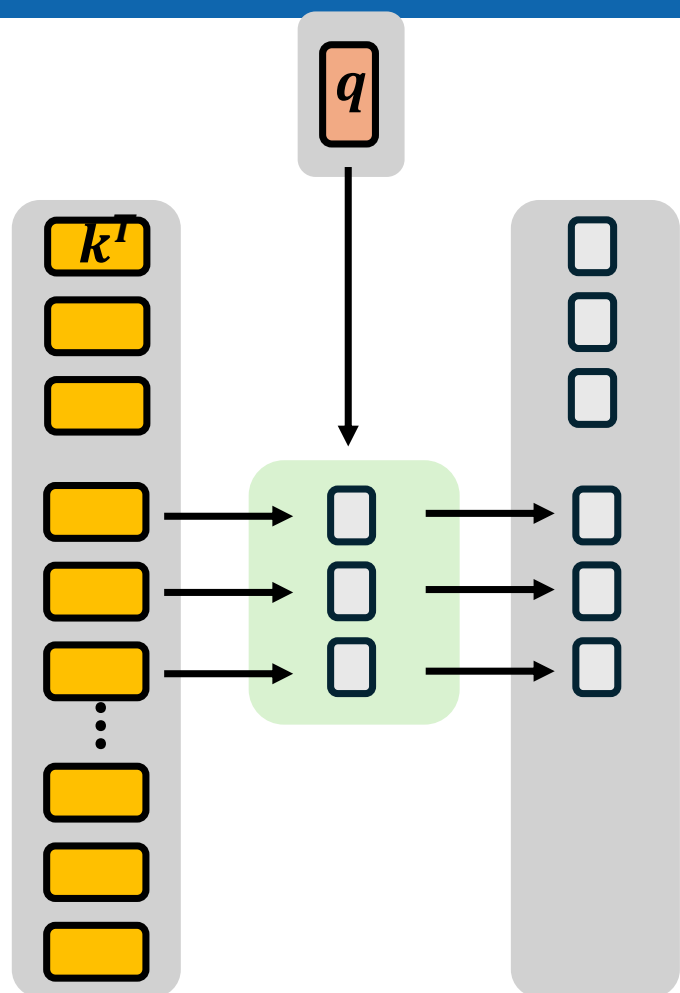
当处理完全部 T 个块后, 最终输出就是 o_T ,
对应于查询 q_1 的注意力输出。

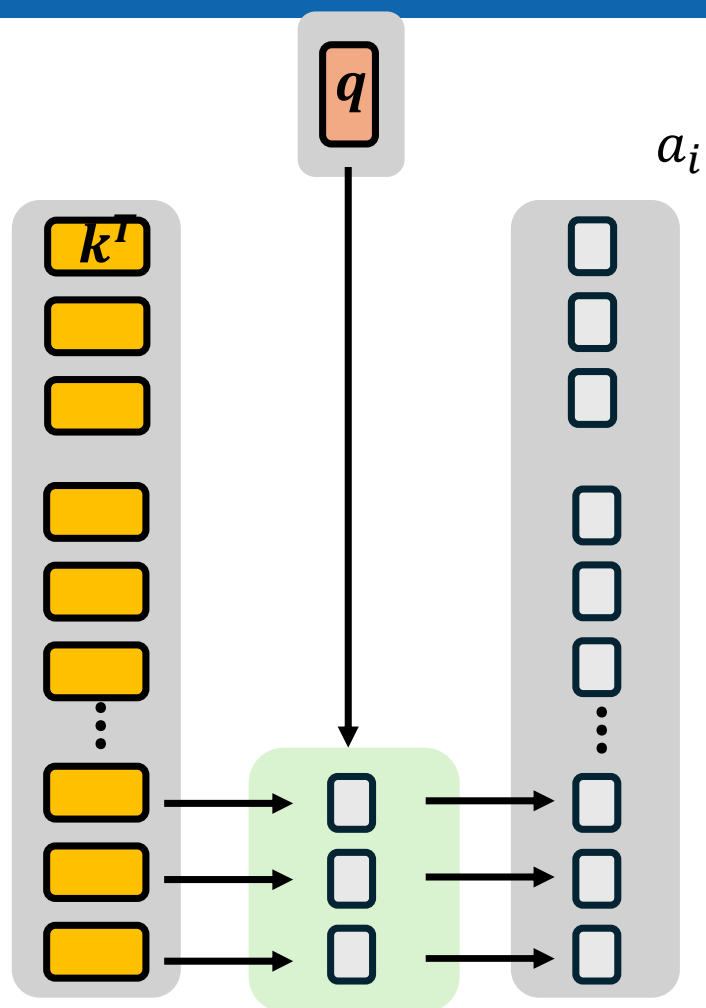


仓库



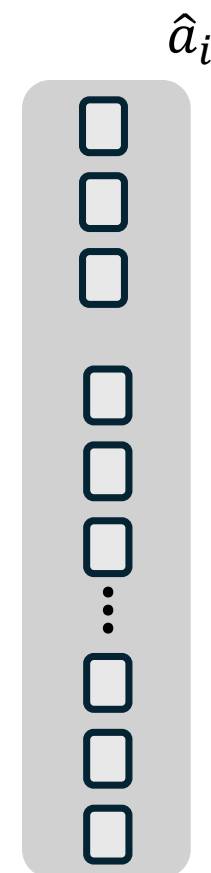
工作台

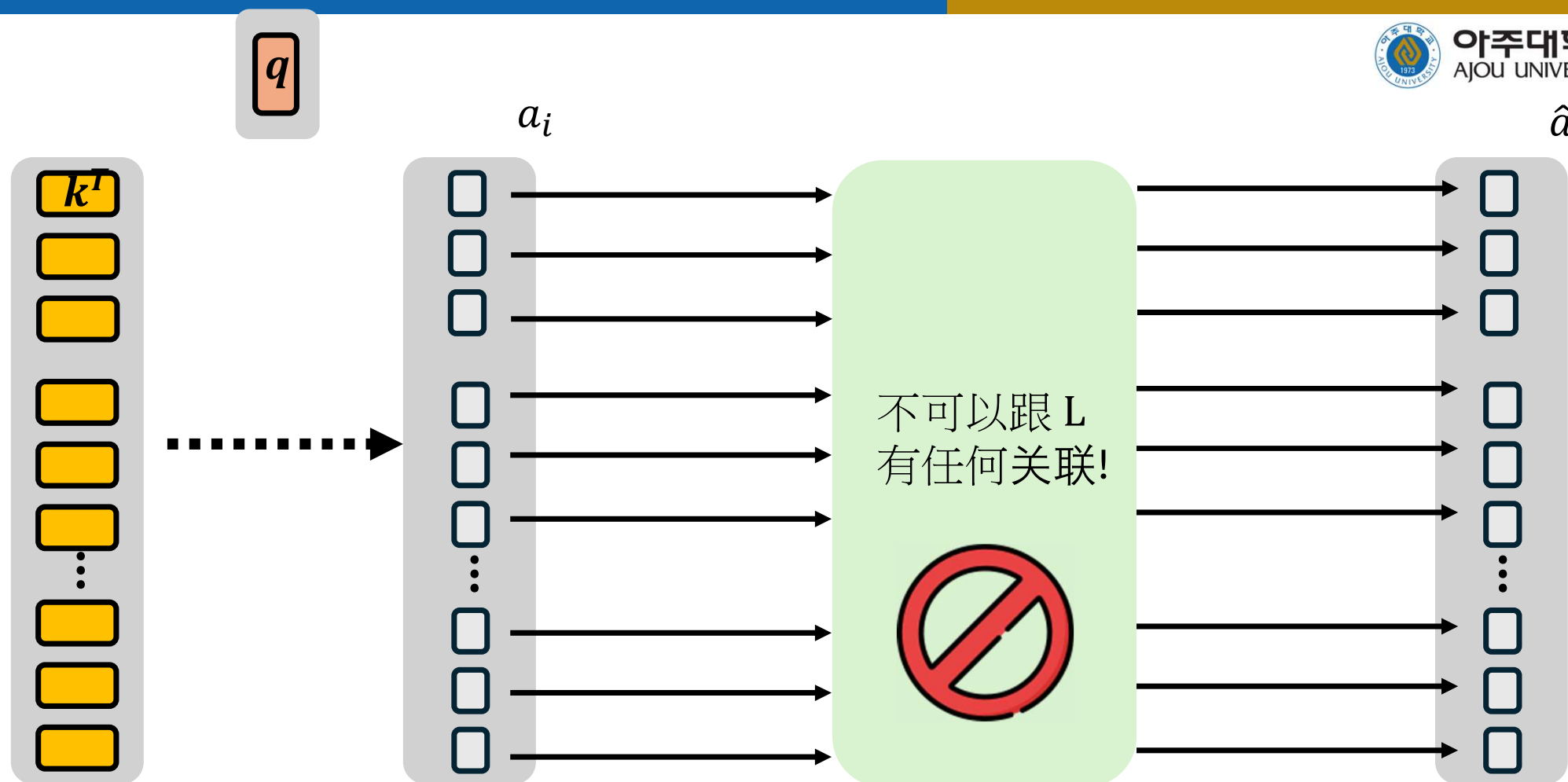




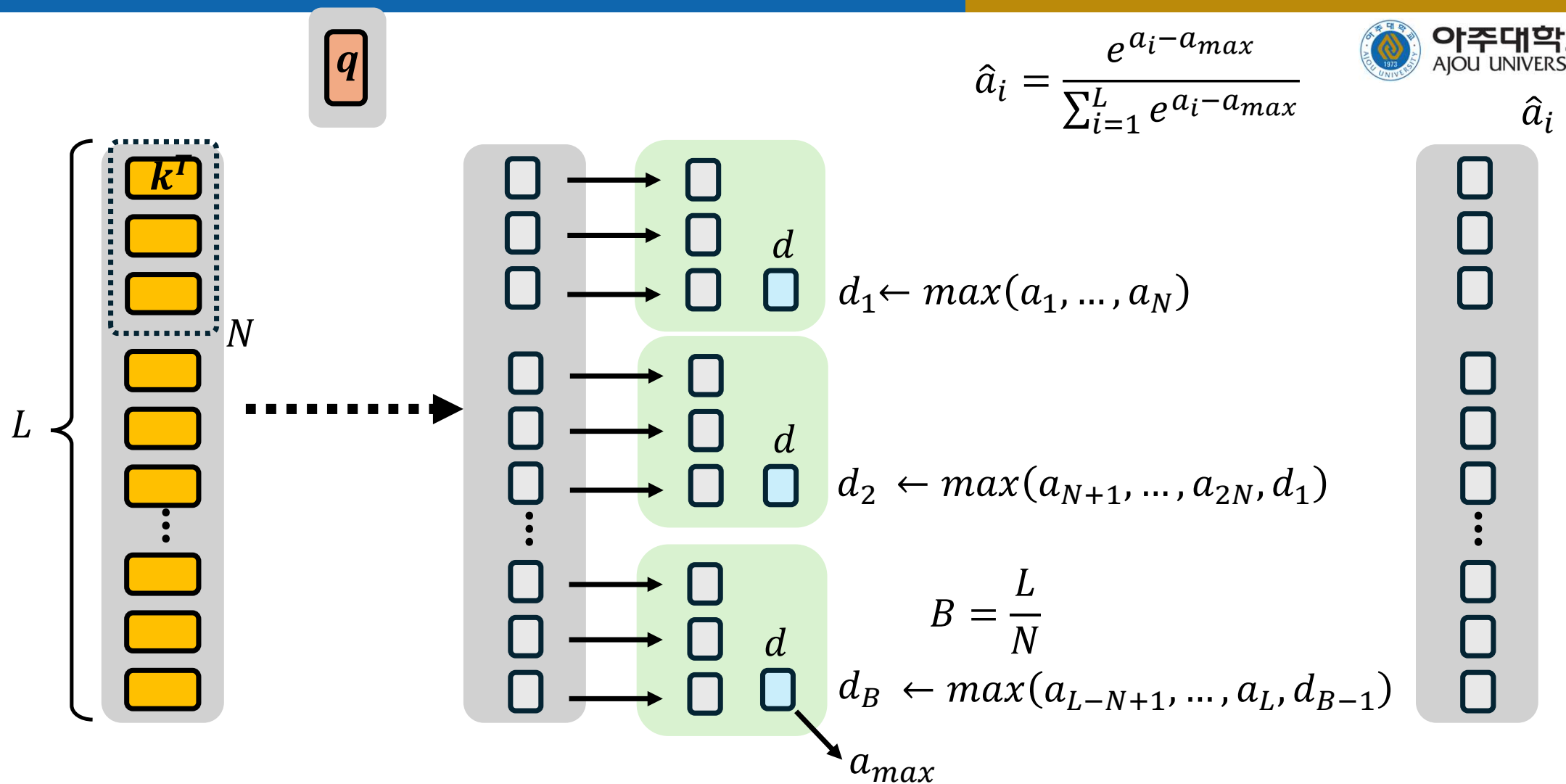
$$\hat{a}_i = \frac{e^{a_i}}{\sum_{i=1}^L e^{a_i}}$$

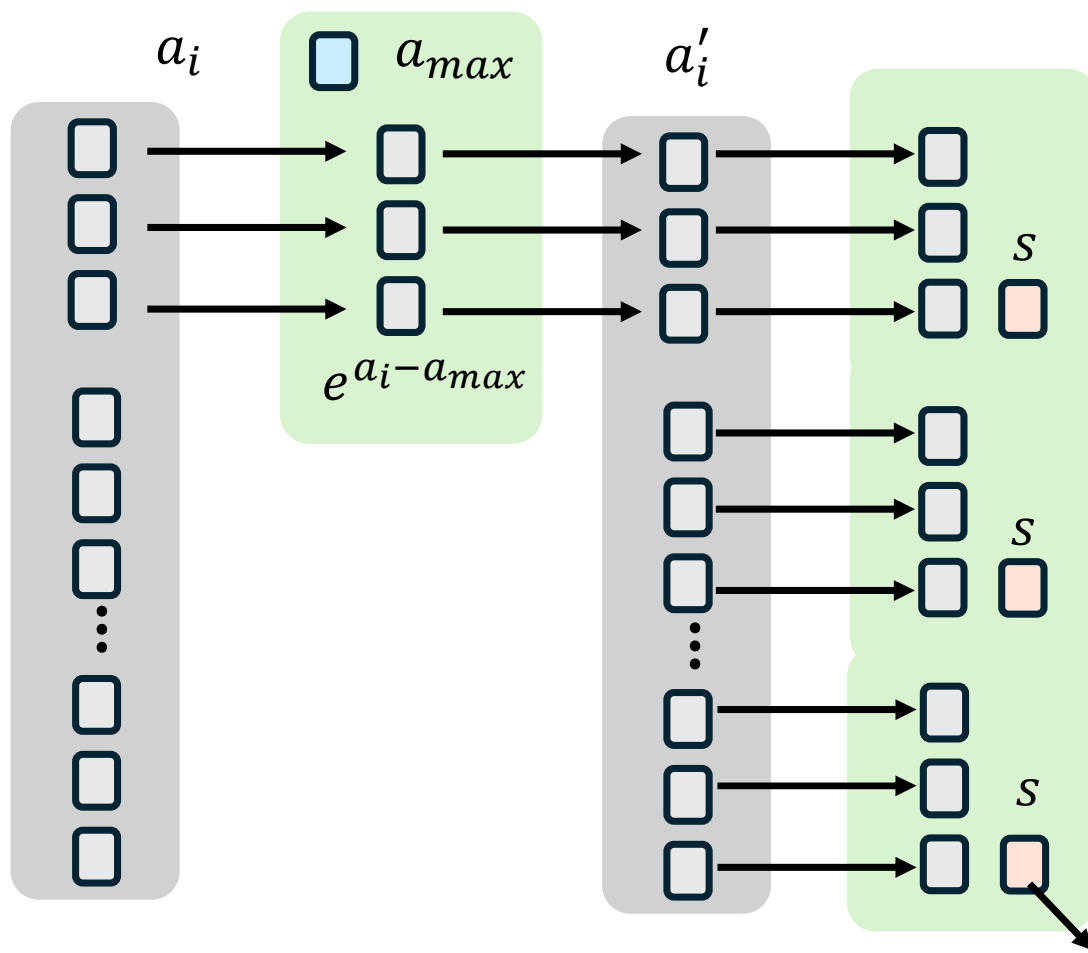
$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}}$$





$$\hat{a}_i = \frac{e^{a_i - a_{\max}}}{\sum_{i=1}^L e^{a_i - a_{\max}}}$$





$$s_1 = \sum_{i=1}^N a'_i$$

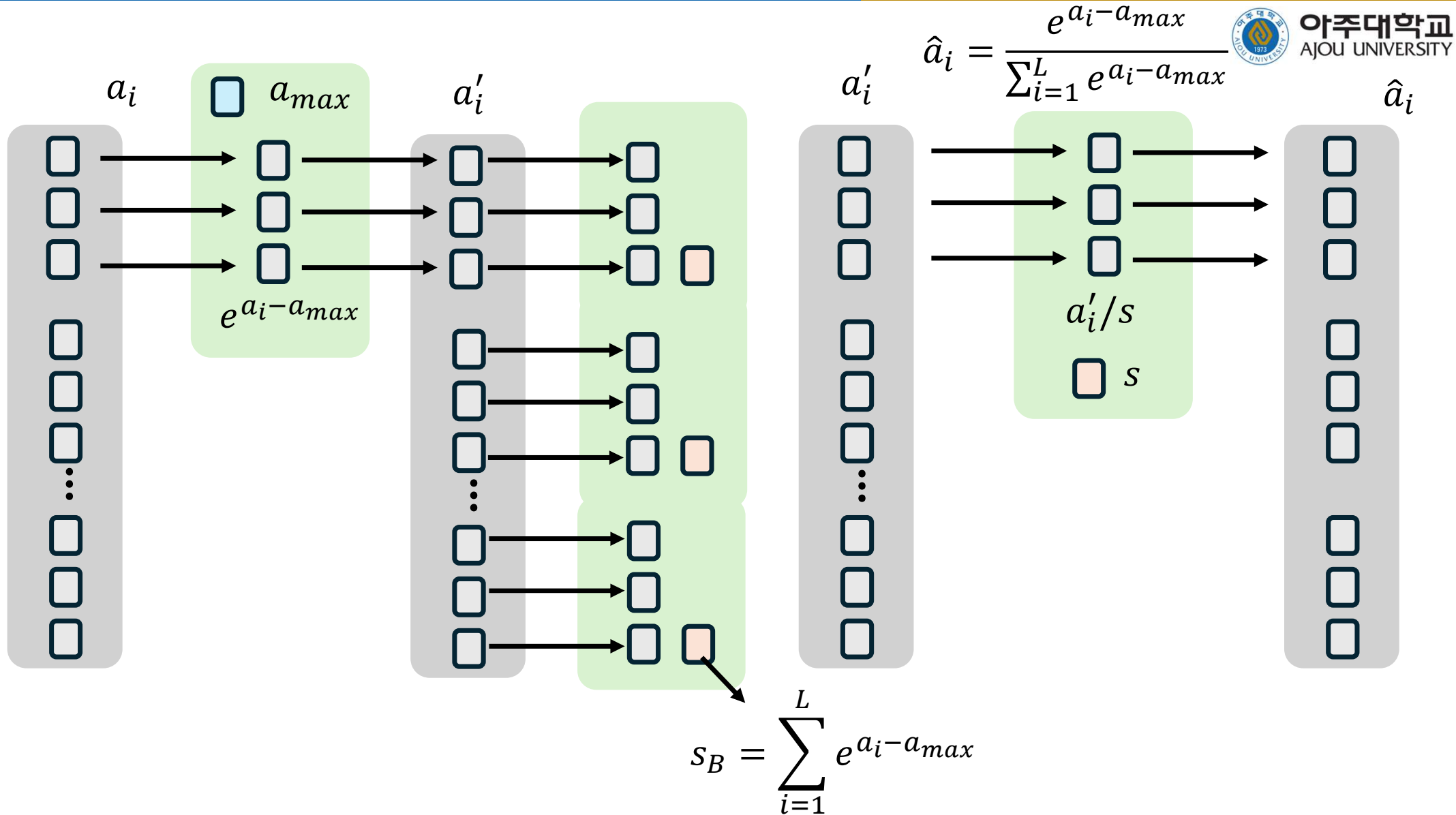
$$s_2 = s_1 + \sum_{i=N+1}^{2N} a'_i$$

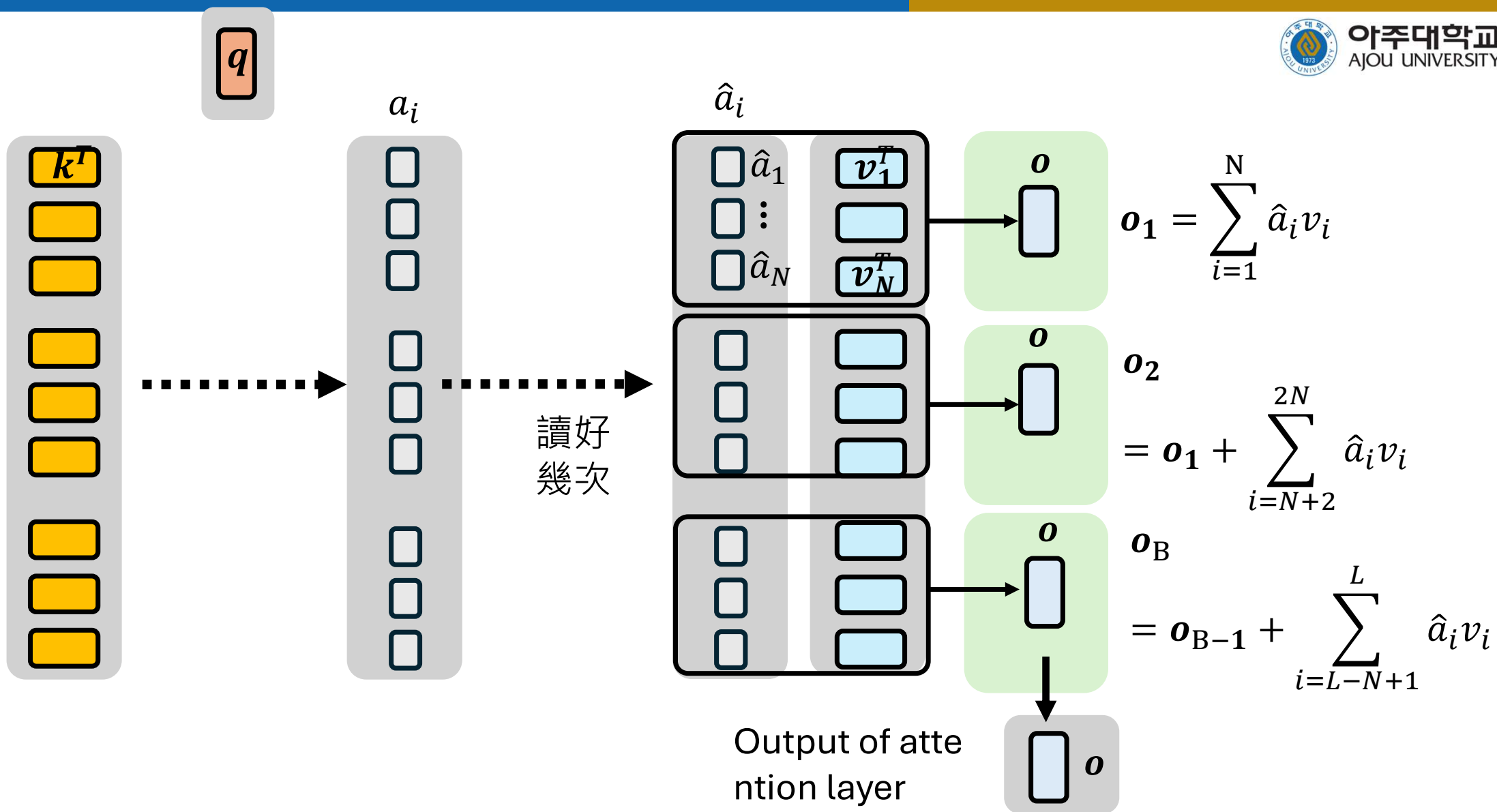
$$s_B = s_{B-1} + \sum_{i=L-N+1}^L a'_i$$

$$\sum_{i=1}^L a'_i = \sum_{i=1}^L e^{a_i - a_{max}}$$

$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}} \hat{a}_i$$

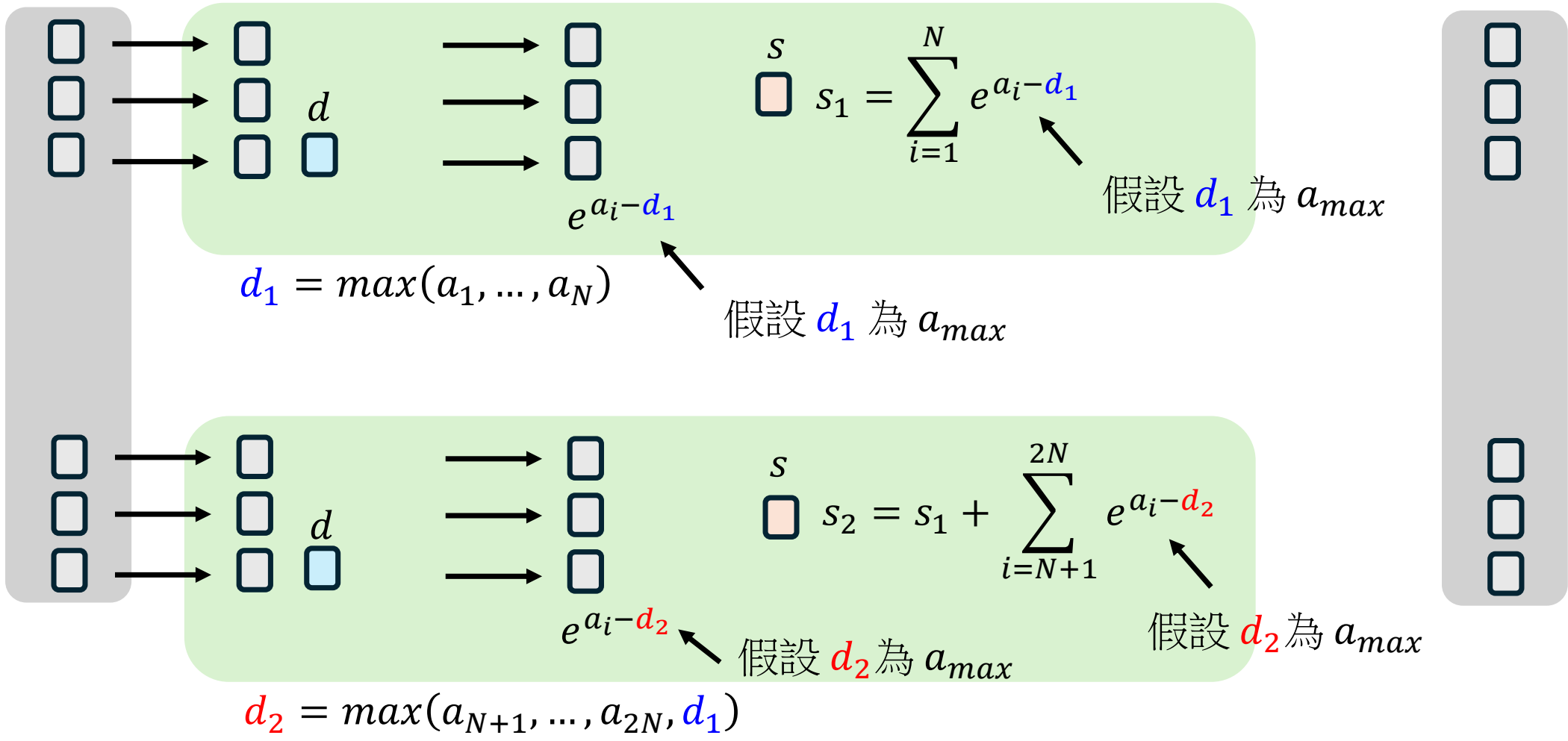






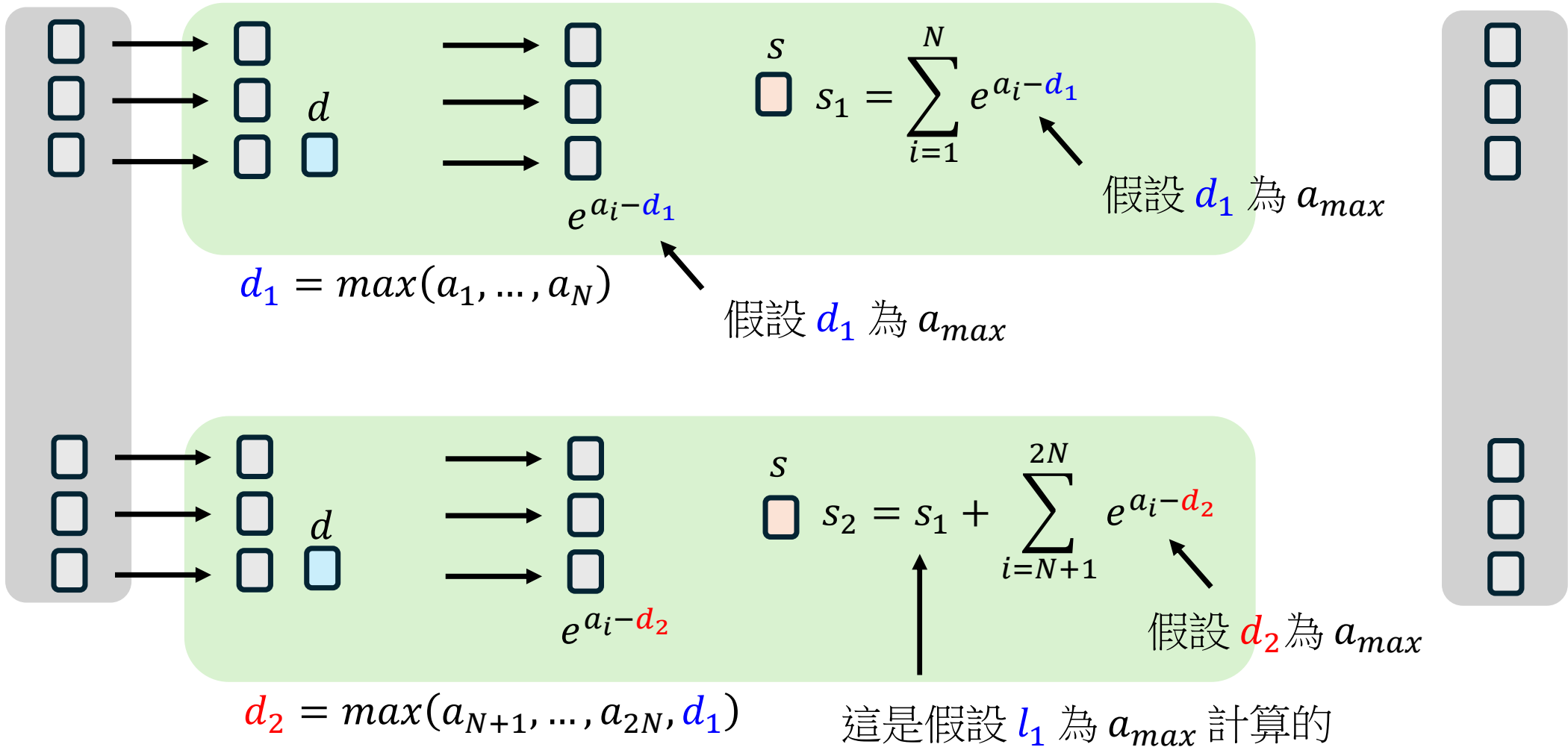
一次找出 a_{max} 和 $\sum_{i=1}^L e^{a_i - a_{max}}$

$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}} \hat{a}_i$$



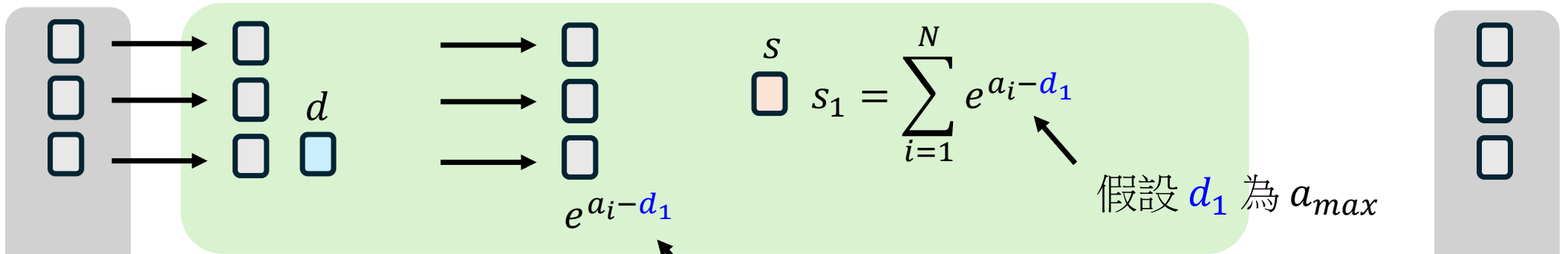
一次找出 a_{max} 和 $\sum_{i=1}^L e^{a_i - a_{max}}$

$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}} \hat{a}_i$$



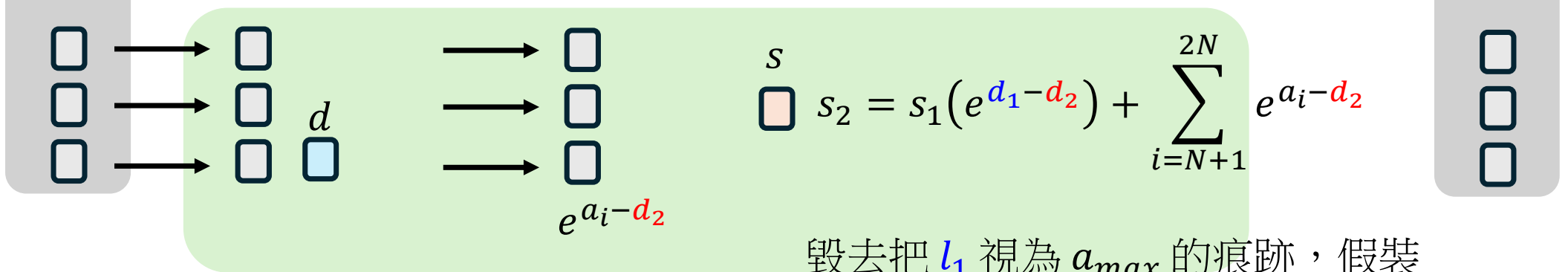
一次找出 a_{max} 和 $\sum_{i=1}^L e^{a_i - a_{max}}$

$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}}$$



$$d_1 = \max(a_1, \dots, a_N)$$

假设 d_1 为 a_{max}

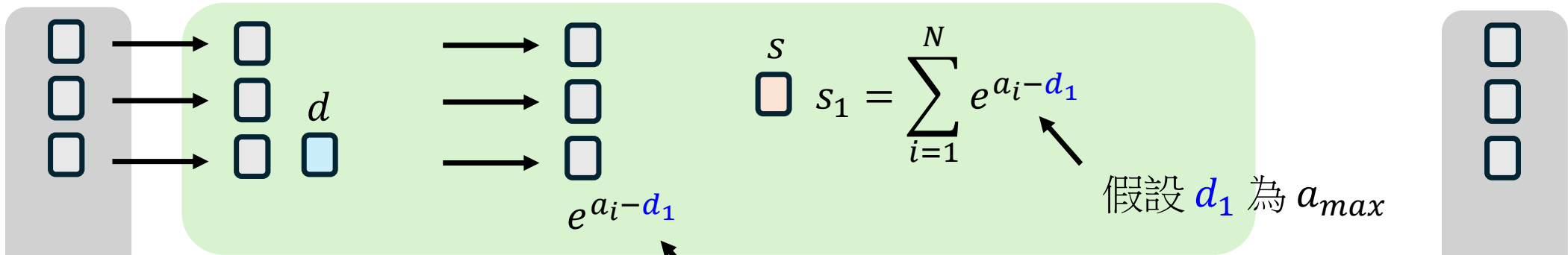


$$d_2 = \max(a_{N+1}, \dots, a_{2N}, d_1)$$

毁灭去把 d_1 视为 a_{max} 的痕迹，假装
 已经是把 d_2 视为 a_{max}

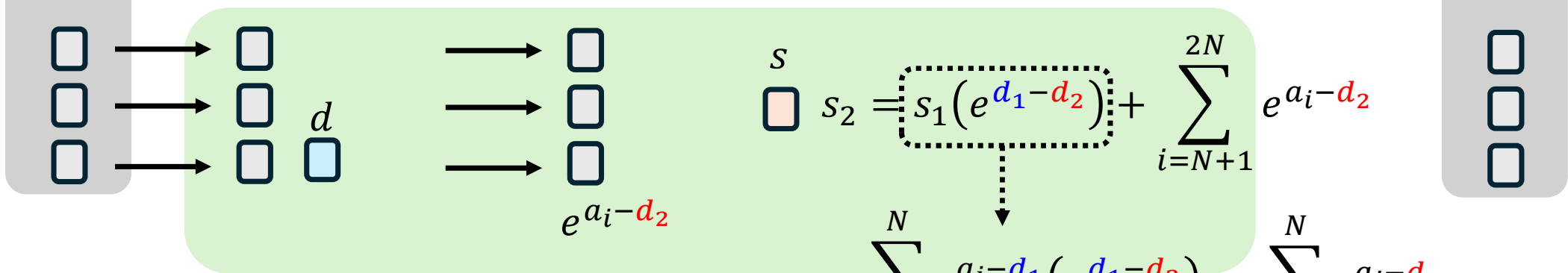
一次找出 a_{max} 和 $\sum_{i=1}^L e^{a_i - a_{max}}$

$$\hat{a}_i = \frac{e^{a_i - a_{max}}}{\sum_{i=1}^L e^{a_i - a_{max}}} \hat{a}_i$$



$$d_1 = \max(a_1, \dots, a_N)$$

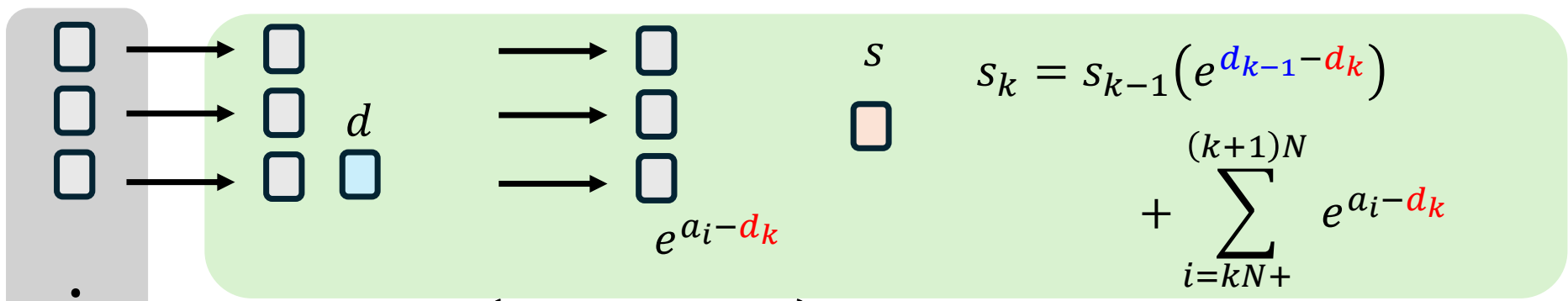
假设 d_1 为 a_{max}



$$d_2 = \max(a_{N+1}, \dots, a_{2N}, d_1)$$

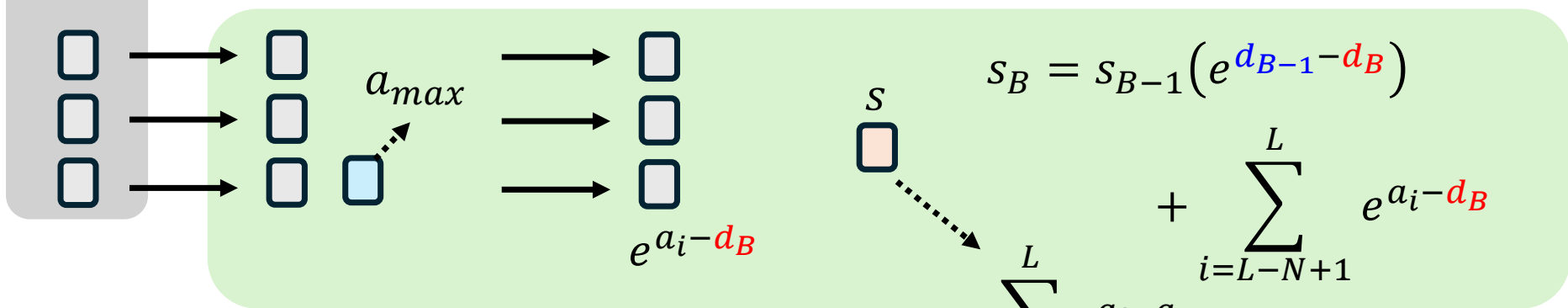
$$\hat{a}_i = \frac{e^{a_i - a_{\max}}}{\sum_{i=1}^L e^{a_i - a_{\max}}} \hat{a}_i$$

k-th chunk



$$d_k = \max(a_1, \dots, a_N, d_{k-1})$$

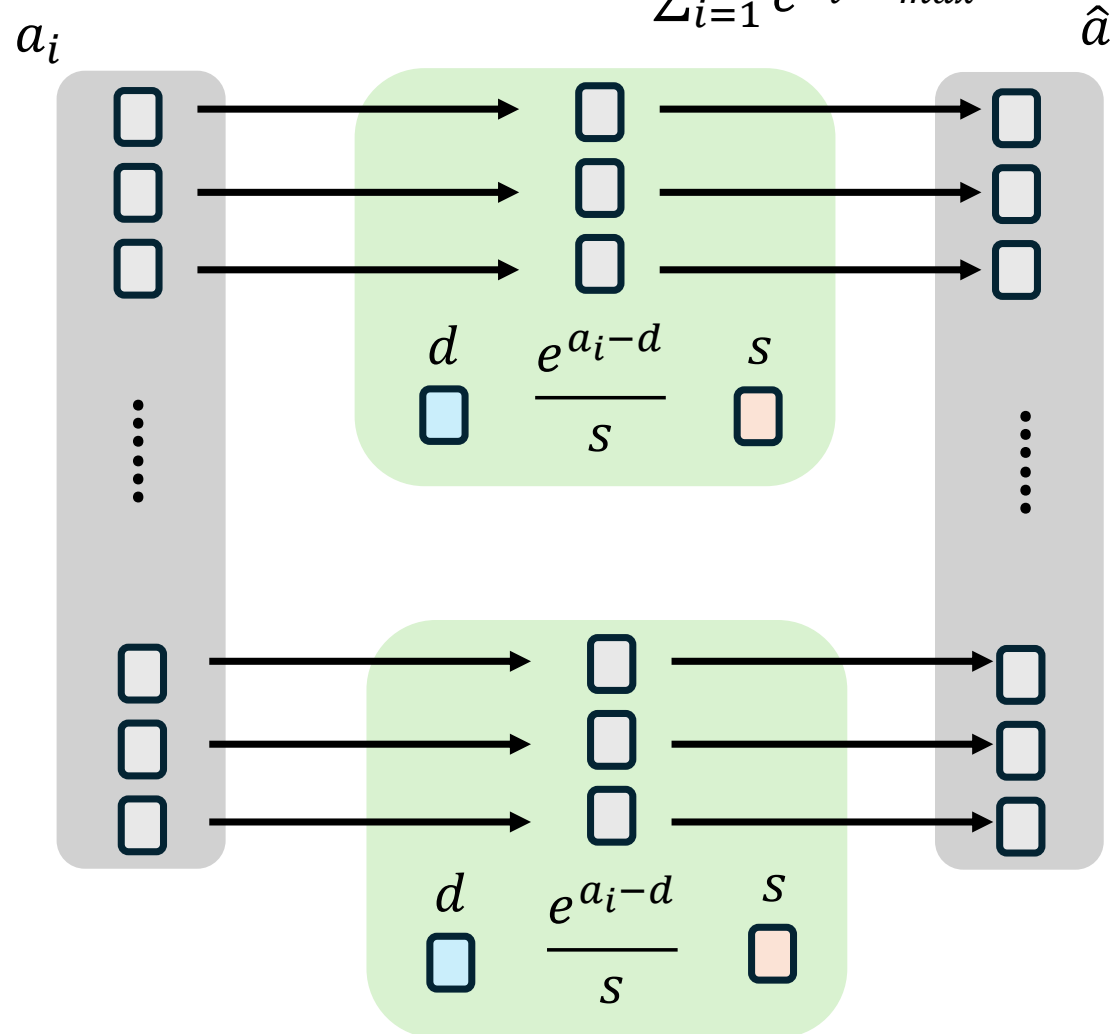
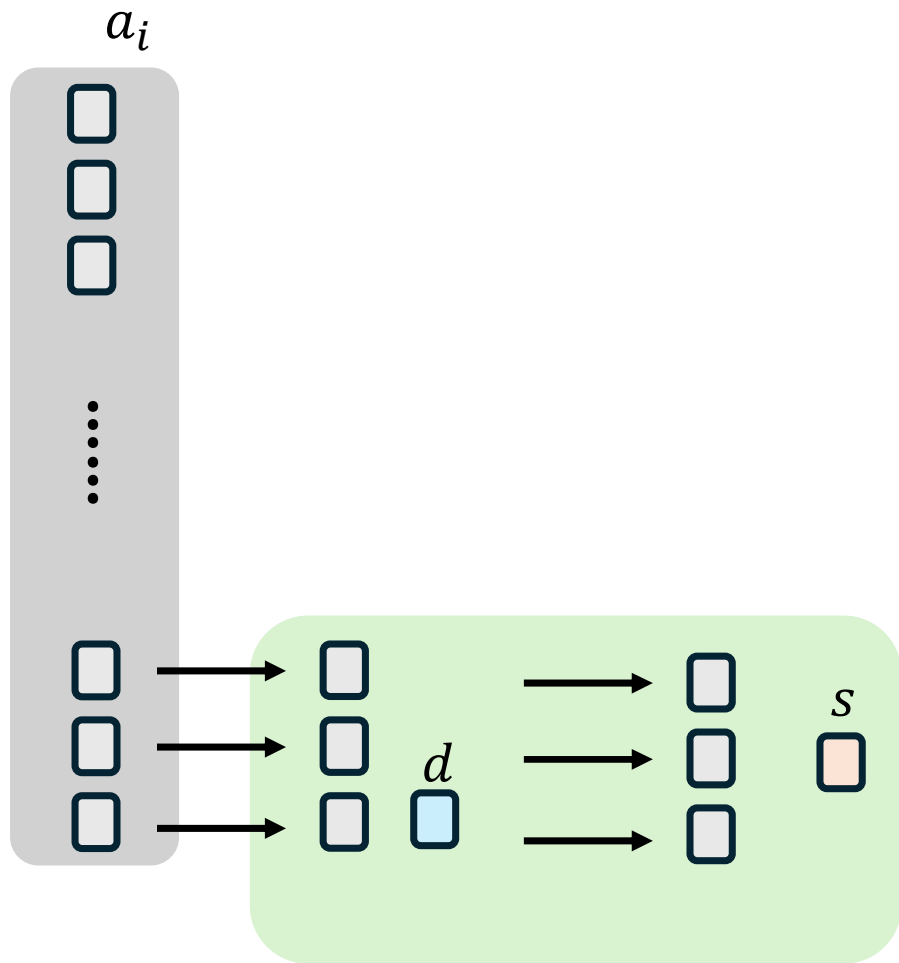
Last chunk (B-th chunk)

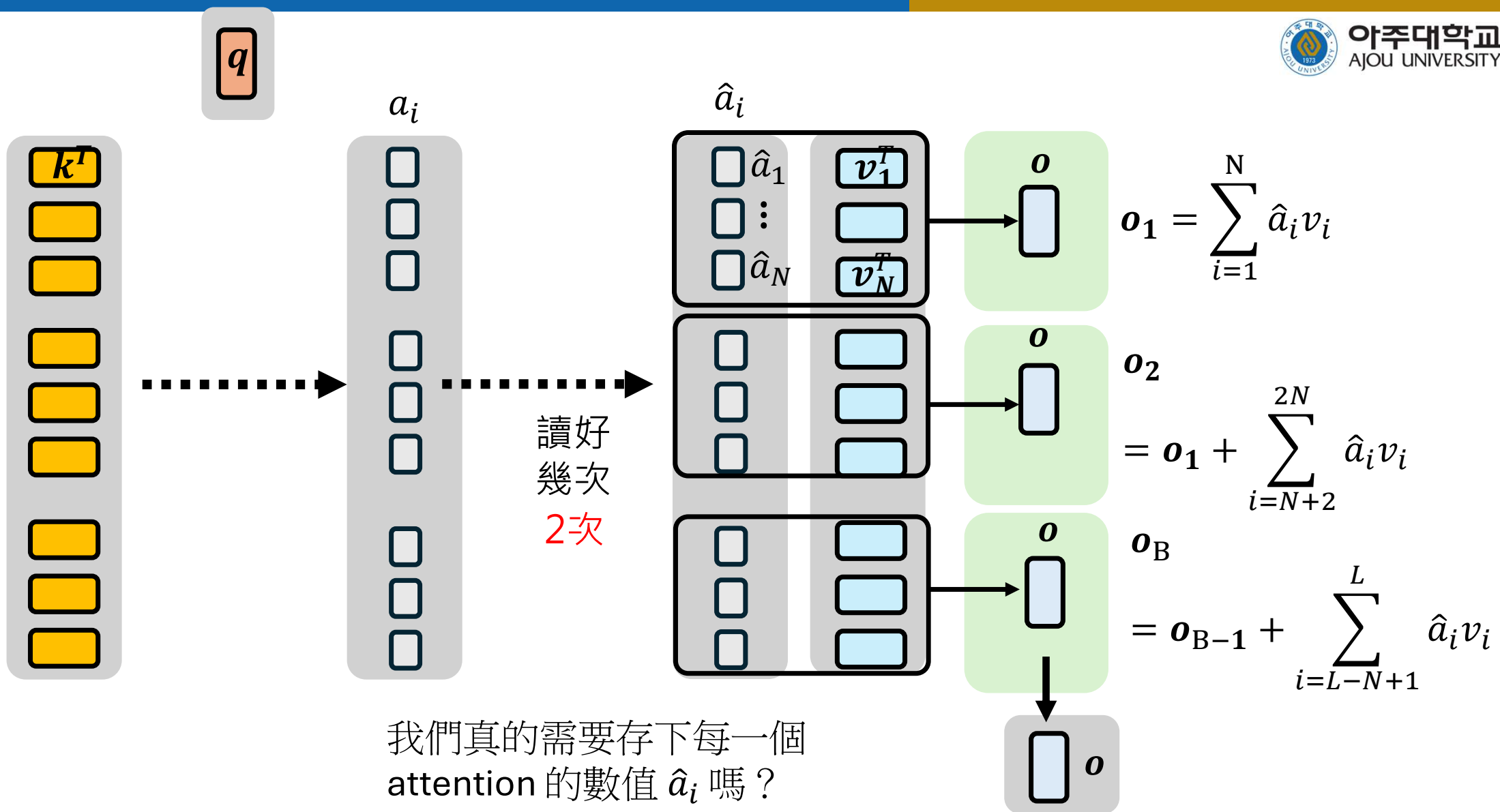


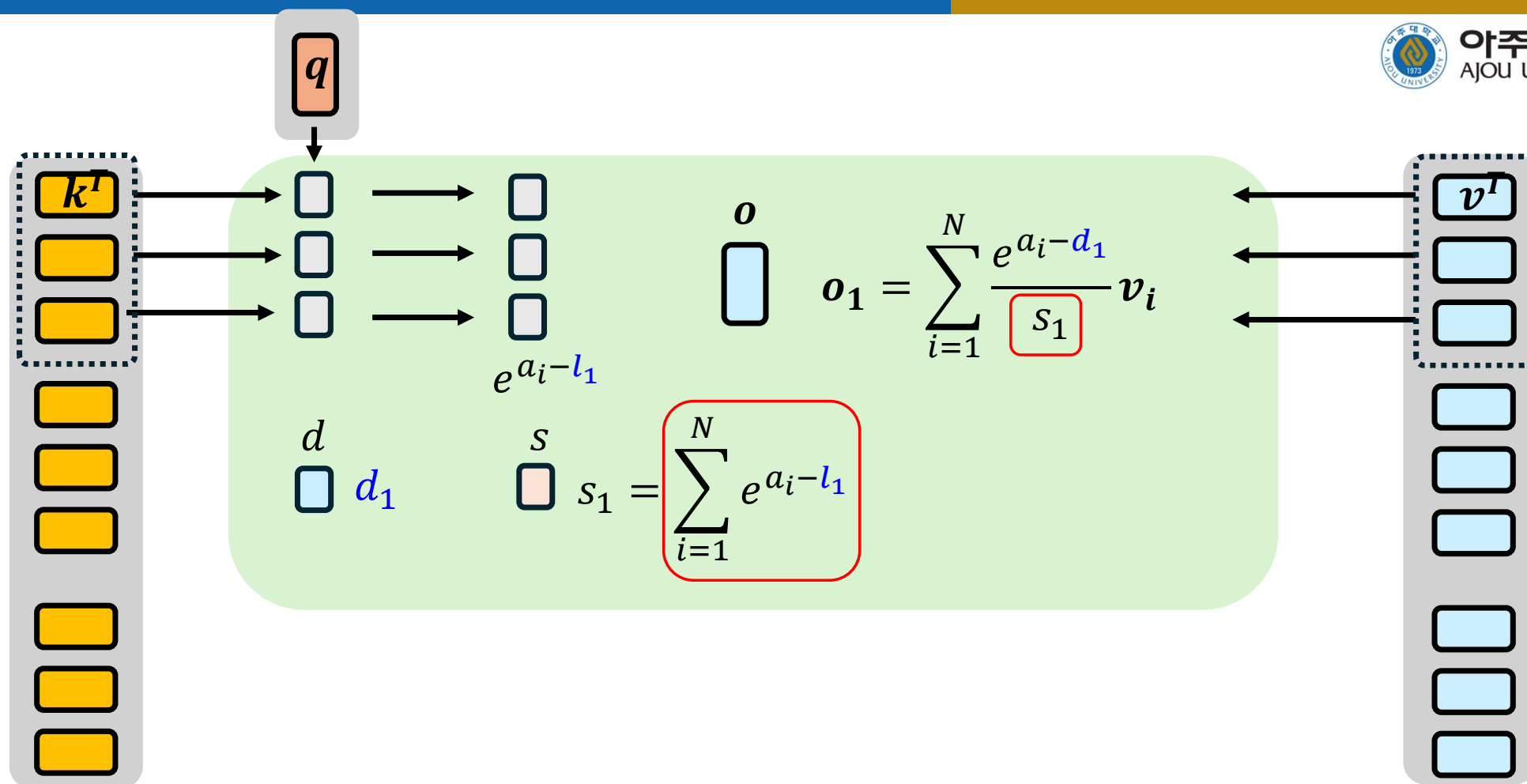
$$d_B = \max(a_{L-N+1}, \dots, a_L, d_{B-1})$$

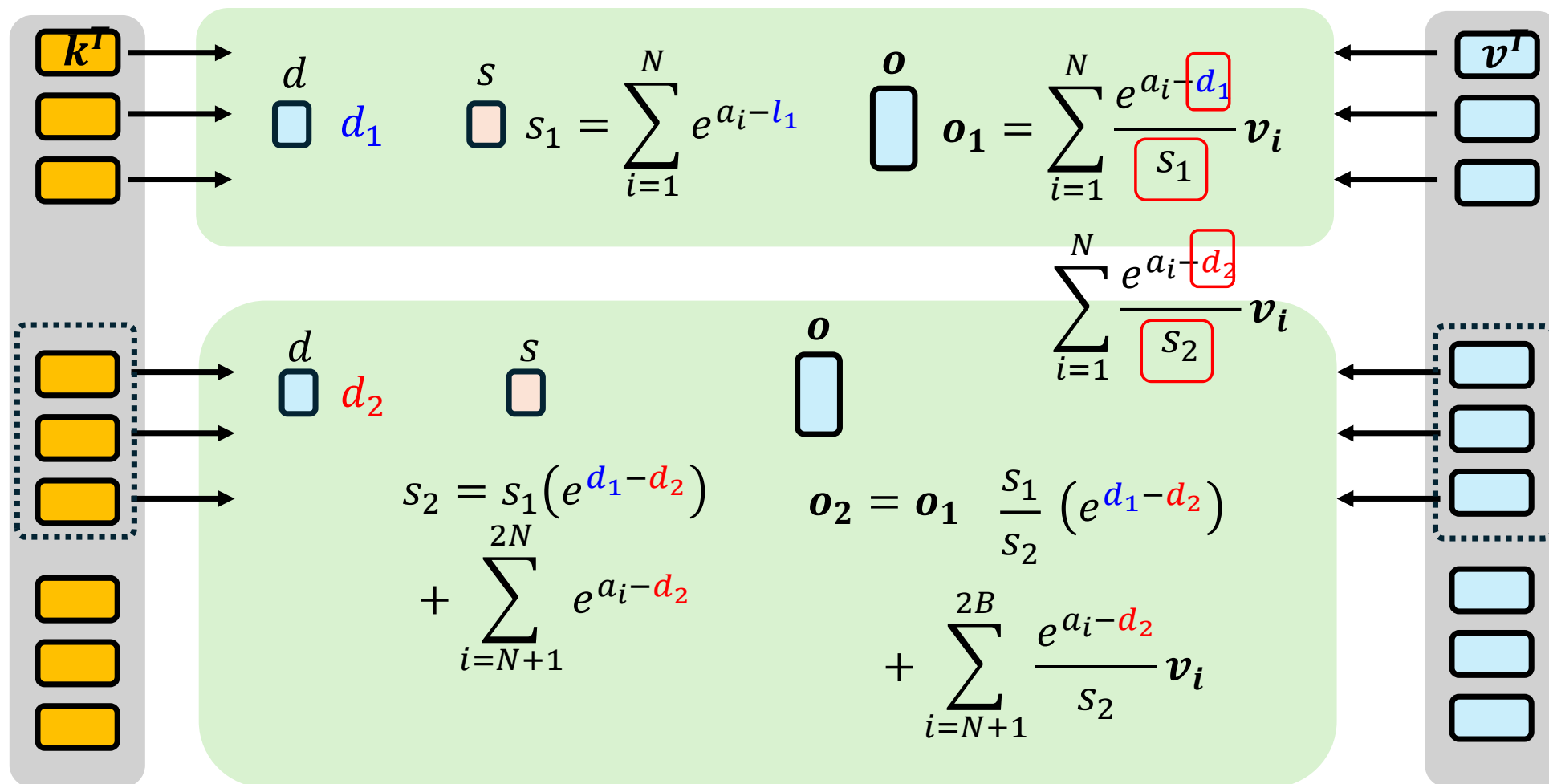
$$\sum_{i=1}^L e^{a_i - a_{\max}}$$

$$\hat{a}_i = \frac{e^{a_i - a_{\max}}}{\sum_{i=1}^L e^{a_i - a_{\max}}}$$

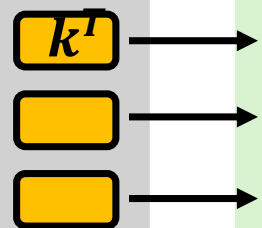






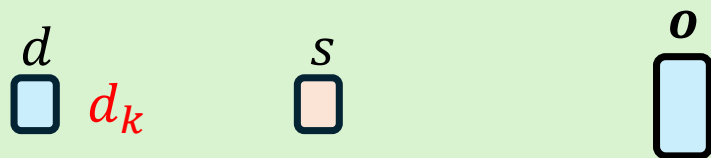
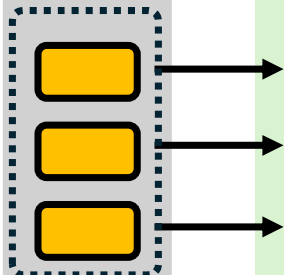


k-th chunk

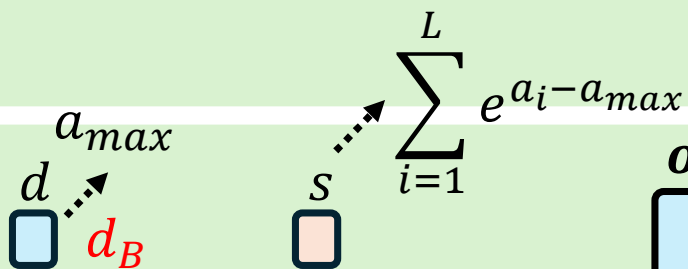


⋮

last chunk
(B-th)



$$s_k = s_{k-1} \left(e^{d_{k-1} - d_k} \right) + \sum_{i=kN+1}^{(k+1)N} e^{a_i - d_k} \quad o_k = o_{k-1} \frac{s_{k-1}}{s_k} \left(e^{d_{k-1} - d_k} \right) + \sum_{i=kN+1}^{(k+1)N} \frac{e^{a_i - d_k}}{s_k} v_i$$

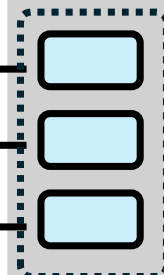


$$s_B = s_{B-1} \left(e^{d_{B-1} - d_B} \right) + \sum_{i=L-N+1}^L e^{a_i - d_B} \quad o_B = o_{B-1} \frac{s_{B-1}}{s_B} \left(e^{d_{B-1} - d_B} \right) + \sum_{i=L-N+1}^L \frac{e^{a_i - d_B}}{s_B} v_i$$

What we want!



⋮



範例程式

- https://colab.research.google.com/drive/1KoeKKIXSXI9b-pYg0kun3-uLQkP6p_hC?usp=sharing

